

JavaScript

- JavaScript is a programming language that allows you to create dynamic and interactive web pages. It is the most popular programming language.
- JavaScript is a case-sensitive language. This means that language keywords, variables, function names, and any other identifiers must always be typed with a consistent capitalization of letters.
- JavaScript is a programming language initially designed to interact with elements of web pages
- JavaScript allows us to add interactivity to a web page. Typically, we use JavaScript with HTML and CSS to enhance a web page's functionality, such as validating forms etc.
- The JavaScript engine is a program that executes JavaScript code. In the beginning, JavaScript engines were implemented as interpreters.
- However, modern JavaScript engines are typically implemented as just-in-time compilers that compile JavaScript code to bytecode for improved performance.

Client-side vs. Server-side JavaScript

When JavaScript is used on a web page, it is executed in web browsers. In this case, JavaScript works as a client-side language.

JavaScript can run on both web browsers and servers. A popular JavaScript server-side environment is Node.js. Unlike client-side JavaScript, server-side JavaScript executes on the server and allows you to access databases, file systems, etc.

Initially when JS was invented by Brendan Eich in 1995, it was supposed to function only as a scripting language on the client side for web development. However, JS has evolved and diversified a lot in many other domains like - Server side programming, Desktop application development, Mobile Application development, Virtual Reality, Machine Learning etc. Here are few Examples of different Technologies & Frameworks where JavaScript is used –

Node.JS: Used in server side programming - complete framework for server side development.

React JS: It's a front-end library developed by Facebook. Using React JS you can create web as well as mobile applications.

React Native: It's a JavaScript framework for writing real, natively rendering mobile applications for iOS and Android.

AngularJS: It's a JavaScript MVC framework to develop a dynamic web application. Started by Google but is open source.

Electron JS: It's a framework used to build desktop apps with JavaScript, HTML, and CSS

How to Add JavaScript to HTML

There are two ways to add JavaScript to HTML: the first is to embed inline JS code into HTML, while the second requires us to create an external JS file.

Embed JavaScript code on an HTML page

Placing JavaScript code inside the `<script>` element directly is not recommended and should be used only for proof of concept or testing purposes.

The JavaScript code in the `<script>` element is interpreted from top to bottom. For example:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>JavaScript Hello World Example</title>

  <script>

    alert('Hello, World!');

  </script>

</head>

<body>
```

```
</body>
```

```
</html>
```

Include an external JavaScript file

To include a JavaScript from an external file:

First, create a file whose extension is .js e.g., app.js and place it in the js subfolder. Note that placing the JavaScript file in the js folder is not required however it is a good practice.

Then, use the URL to the JavaScript source code file in the src attribute of the `<script>` element.

The following shows the contents of the app.js file:

```
alert('Hello, World!');
```

Code language: JavaScript (javascript)

And the following shows the helloworld.html file:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>JavaScript Hello World Example</title>
```

```
  <script src="js/app.js"></script>
```

```
</head>
```

```
<body>
```

```
</body>
```

Whitespace

Whitespace refers to characters that provide the space between other characters. JavaScript has the following whitespace:

- Carriage return
- Space
- New Line
- tab

JavaScript engine ignores whitespace. However, you can use whitespace to format the code to make it easy to read and maintain.

The following JavaScript code doesn't use whitespace:

```
let formatted = true; if (formatted) {console.log('The code is easy to read');}
```

It is equivalent to the following code that uses whitespace. Hence, this code is much easier to read:

```
let formatted = true;
```

```
if (formatted) {  
  console.log('The code is easy to read');  
}
```

Note that JavaScript bundlers remove all whitespace from JavaScript files and put them into a single file for deployment. By doing this, JavaScript bundlers make the JavaScript code lighter and faster to load in web browsers.

Statements

A statement is a code that declares a variable or instructs the JavaScript engine to do a task. A simple statement is terminated by a semicolon (;).

Although the semicolon (;) is optional; you should always use it to terminate a statement. For example, the following declares a variable and shows it to the console:

```
let message = "Welcome to JavaScript";  
console.log(message);
```

Blocks

A block is a sequence of zero or more simple statements. A block is delimited by a pair of curly brackets {}. For example:

```
if (condition) {  
    console.log(statement);  
}
```

Comments

Comments allow you to add notes or hints to JavaScript code. When executing the code, the JavaScript engine ignores the comments.

JavaScript supports single-line and block comments.

Single-line comments

A single-line comment starts with two forward-slashes characters (//). A single-line comment makes all the text following the // on the same line into a comment. For example:

```
// this is a single-line comment
```

Block comments

A delimited comment begins with a forward slash and asterisk /* and ends with the opposite */ as in the following example:

```
/* This is a block comment  
that can span multiple lines */
```

Expressions

An expression is a piece of code that evaluates to a value. For example:

```
2 + 1
```

The above expression returns three.

JavaScript Variables

A variable is a label that references a value like a number or string. Before using a variable, we need to declare it.

Declare a variable

To declare a variable, we use the `var` keyword followed by the variable name as follows:

```
var message;
```

A variable name can be any valid identifier. By default, the message variable has a special value `undefined` if you have not assigned a value to it.

Variable names follow these rules:

- Variable names are case-sensitive. This means that the `message` and `Message` are different variables.
- Variable names can only contain letters, numbers, underscores, or dollar signs and cannot contain spaces. Also, variable names must begin with a letter, an underscore (`_`) or a dollar sign (`$`).
- Variable names cannot use the reserved words.

By convention, variable names use camelcase like `message`, `yourAge`, and `myName`.

JavaScript is a dynamically typed language. This means that we don't need to specify the variable's type in the declaration like other static-typed languages such as Java or C#.

Starting in ES6, we can use the `let` keyword to declare a variable like this:

```
let message;
```

It's a good practice to use the `let` keyword to declare a variable.

Initialize a variable

Once we declared a variable, we can initialize it with a value. To initialize a variable, we specify the variable name, followed by an equals sign (`=`) and a value.

For example, The following declares the message variable and initializes it with a literal string `"Hello"`:

```
let message;
```

```
message = "Hello";
```

To declare and initialize a variable at the same time, we use the following syntax:

```
let variableName = value;
```

For example, the following statement declares the message variable and initializes it with the literal string "Hello":

```
let message = "Hello";
```

JavaScript allows us to declare two or more variables using a single statement. To separate two variable declarations, you use a comma (,) like this:

```
let message = "Hello", counter = 100;
```

Since JavaScript is a dynamically typed language, we can assign a value of a different type to a variable. Although, it is not recommended. For example:

```
let message = 'Hello';
```

```
message = 100;
```

Change a variable

Once you initialize a variable, you can change its value by assigning a different value. For example:

```
let message = "Hello";
```

```
message = 'Bye';
```

Code language: JavaScript (javascript)

Undefined vs. undeclared variables

It's important to distinguish between undefined and undeclared variables.

An undefined variable is a variable that has been declared but has not been initialized with a value. For example:

```
let message;
```

```
console.log(message); // undefined
```

In this example, the message variable is declared but not initialized. Therefore, the message variable is undefined.

In contrast, an undeclared variable is a variable that has not been declared. For example:

```
console.log(counter);
```

Code language: JavaScript (javascript)

Output:

```
console.log(counter);
```

Constants

A constant holds a value that doesn't change. To declare a constant, we use the `const` keyword. When defining a constant, we need to initialize it with a value. For example:

```
const workday = 5;
```

Once we define a constant, we cannot change its value.

The following example attempts to change the value of the `workday` constant to 4 and causes an error:

```
workday = 2;
```

Error:

Uncaught TypeError: Assignment to constant variable.

JavaScript Data Types

JavaScript has the primitive data types:

1. `null`
2. `undefined`
3. `boolean`
4. `number`
5. `string`
6. `symbol` – available from ES2015
7. `bigint` – available from ES2020

JavaScript is a dynamically typed language. It means that a variable doesn't associate with a type. In other words, a variable can hold a value of different types. For example:

```
let counter = 120; // counter is a number  
counter = false; // counter is now a boolean  
counter = "foo"; // counter is now a string
```

To get the current type of the value that the variable stores, we will use the `typeof` operator:

```
let counter = 120;  
console.log(typeof(counter)); // "number"  
counter = false;  
console.log(typeof(counter)); // "boolean"  
counter = "Hi";  
console.log(typeof(counter)); // "string"
```

The undefined type

The undefined type is a primitive type that has only one value undefined. By default, when a variable is declared but not initialized, it is assigned the value of undefined.

Consider the following example:

```
let counter;  
console.log(counter);    // undefined  
console.log(typeof counter); // undefined
```

In this example, the counter is a variable. Since counter hasn't been initialized, it is assigned the value undefined. The type of counter is also undefined.

It's important to note that the `typeof` operator also returns undefined when you call it on a variable that hasn't been declared:

```
console.log(typeof undeclaredVar); // undefined
```

The null type

The null type is the second primitive data type that also has only one value null. For example:

```
let obj = null;  
  
console.log(typeof obj); // object
```

The `typeof null` returns `object` is a known bug in JavaScript. A proposal to fix this was proposed but rejected. The reason was that the fix would break a lot of existing sites.

JavaScript defines that null is equal to undefined as follows:

```
console.log(null == undefined); // true
```

Code language: JavaScript (javascript)

The number type

JavaScript uses the number type to represent both integer and floating-point numbers.

The following statement declares a variable and initializes its value with an integer:

```
let num = 100;
```

To represent a floating-point number, you include a decimal point followed by at least one number. For example:

```
let price = 12.5;  
  
let discount = 0.05;
```

Note that JavaScript automatically converts a floating-point number into an integer number if the number appears to be a whole number.

The reason is that JavaScript always wants to use less memory since a floating-point value uses twice as much memory as an integer value. For example:

```
let price = 200.00; // interpreted as an integer 200
```

To get the range of the number type, we use `Number.MIN_VALUE` and `Number.MAX_VALUE`. For example:

```
console.log(Number.MAX_VALUE); // 1.7976931348623157e+308
```

```
console.log(Number.MIN_VALUE); // 5e-324
```

Also, we can use Infinity and -Infinity to represent the infinite number. For example:

```
console.log(Number.MAX_VALUE + Number.MAX_VALUE); // Infinity
```

```
console.log(-Number.MAX_VALUE - Number.MAX_VALUE); // -Infinity
```

NaN

NaN stands for Not a Number. It is a special numeric value that indicates an invalid number. For example, the division of a string by a number returns NaN:.

```
console.log('a'/2); // NaN;
```

The NaN has two special characteristics:

Any operation with NaN returns NaN.

The NaN does not equal any value, including itself.

Here are some examples:

```
console.log(NaN/2); // NaN
```

```
console.log(NaN == NaN); // false
```

The string type

In JavaScript, a string is a sequence of zero or more characters. A string literal begins and ends with either a single quote(') or a double quote (").

A string that begins with a double quote must end with a double quote.

Likewise, a string that begins with a single quote must also end with a single quote:

```
let greeting = 'Hi';
```

```
let message = "Bye";
```

If we want to single quote or double quotes in a literal string, we need to use the backslash to escape it. For example:

```
let message = 'I\'m also a valid string'; // use \ to escape the single quote (')
```

JavaScript strings are immutable. This means that it cannot be modified once created. However, we can create a new string from an existing string. For example:

```
let str = 'JavaScript';
```

```
str = str + ' String';
```

In this example:

First, declare the str variable and initialize it to a string of 'JavaScript'.

Second, use the + operator to combine 'JavaScript' with ' String' to make its value as 'Javascript String'.

Behind the scene, the JavaScript engine creates a new string that holds the new string 'JavaScript String' and destroys the original strings 'JavaScript' and ' String'.

The following example attempts to change the first character of the string JavaScript:

```
let s = 'JavaScript';
```

```
s[0] = 'j';
```

```
console.log(s)
```

The output is:

```
'JavaScript'
```

But not:

```
'javaScript'
```

The boolean type

The boolean type has two literal values: true and false in lowercase. The following example declares two variables that hold the boolean values.

```
let inProgress = true;
```

```
let completed = false;
```

```
console.log(typeof completed); // boolean
```

JavaScript allows values of other types to be converted into boolean values of true or false.

To convert a value of another data type into a boolean value, we use the Boolean() function. The following table shows the conversion rules:

Type true false

string non-empty string empty string

number non-zero number and Infinity 0, NaN

object non-null object null

undefined undefined

For example:

```
console.log(Boolean('Hi')); // true
```

```
console.log(Boolean('')); // false
```

```
console.log(Boolean(20)); // true
```

```
console.log(Boolean(Infinity)); // true
```

```
console.log(Boolean(0)); // false
```

```
console.log(Boolean({foo: 100})); // true on non-empty object
```

```
console.log(Boolean(null)); // false
```

The symbol type

JavaScript added a primitive type in ES6: the symbol. Different from other primitive types, the symbol type does not have a literal form.

To create a symbol, we call the Symbol function as follows:

```
let s1 = Symbol();
```

The Symbol function creates a new unique value every time we call it.

```
console.log(Symbol() == Symbol()); // false
```

The bigint type

The bigint type represents the whole numbers that are larger than $2^{53} - 1$. To form a bigint literal number, we can append the letter n at the end of the number:

```
let pageView = 9007199254740991n;
```

```
console.log(typeof(pageView)); // 'bigint'
```

Console log()

All modern browsers have a web console for debugging. The `console.log()` method is used to write messages to these consoles. For example,

```
let sum = 44;
```

```
console.log(sum); // 44
```

`console.log()` Syntax

Its syntax is:

```
console.log(message);
```

Here, the message refers to either a variable or a value.

Example 2: Print Values Stored in Variables

```
// program to print variables values
```

```
// storing values
```

```
const greet = 'Hello';
```

```
const name = 'Jack';
```

```
console.log(greet + ' ' + name);
```

HTML DOM Document write()

The `write()` method writes directly to an open (HTML) document stream.

Write some text directly to the HTML output:

```
document.write("Hello World!");
```

Write some HTML elements directly to the HTML output:

```
document.write("<h2>Hello World!</h2><p>Have a nice day!</p>");
```

Identifier

An identifier is a sequence of characters in the code that identifies a variable, function, or property. In most languages, identifiers are case-sensitive and not quoted.

Rules for Naming JavaScript Identifiers

- In JavaScript, identifiers can contain letters(A-Z,a-z),\$ _ and digits (0-9), but may not start with a digit.
- Identifier names must start with either a letter, an underscore _, or the dollar sign \$.
- Identifier names cannot start with numbers. For example,
- JavaScript is case-sensitive. So y and Y are different identifiers.
- Identifiers cannot contain blank space

JavaScript Keywords

Keywords are reserved words that are part of the syntax in the programming language.

- Keywords cannot be used as identifier names.

JavaScript Variables

To define a variable in JavaScript, you use var keyword. For example:

```
var x = 10;
```

```
var y = 20;
```

Code language: JavaScript (javascript)

ES6 added a new way to declare a variable with the let keyword:

```
let x = 10;
```

```
let y = 20;
```

There are differences between var and let. And it's a good practice to use the let keyword to declare variables.

Variables are Containers for Storing Data

JavaScript Variables can be declared in 4 ways:

- Automatically
- Using var
- Using let
- Using const

In this first example, x, y, and z are undeclared variables.

They are automatically declared when first used:

Example

```
x = 5;
```

```
y = 6;
```

```
z = x + y;
```

It is considered good programming practice to always declare variables before use.

The var keyword was used in all JavaScript code from 1995 to 2015.

The let and const keywords were added to JavaScript in 2015.

The var keyword should only be used in code written for older browsers.

Example using var

```
var x = 5;
```

```
var y = 6;
```

```
var z = x + y;
```

When to Use var, let, or const?

1. Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers.

Difference between var and let in JavaScript

In the early days of JavaScript, there was only one way of declaring variables and that was using the `var` keyword. A variable declared with `var` is defined throughout the program. One of the issues with using the `var` keyword was redeclaring a variable inside a block will also redeclare the variable outside the block. Latest JavaScript come with two more keywords, `let` and `const` came into the picture. `var` and `let` are both used for variable declaration in JavaScript but the difference between them is that `var` is function scoped and `let` is block scoped.

Prefer not to use `var` because of issue in block scope and function scope

```
let accountState; // if we don't define value of the variable then value is undefined
```

Undefined Vs Null in JavaScript

Null in JavaScript means an empty value and is also a primitive type in JavaScript. The variable which has been assigned as null contains no

value. Undefined, on the other hand, means the variable has been declared, but its value has not been assigned. Null is an object

JavaScript typeof Operator

The typeof operator returns the type of variables and values. For example,

```
const a = 9;
console.log(typeof a); // number
console.log(typeof '9'); // string
console.log(typeof false); // boolean
let accountId=123
document.write(typeof accountId)
console.log(typeof null)
console.log(typeof(accountId)); //we can use as a typeof as method
```

Number

The Number constructor contains constants and methods for working with numbers. Values of other types can be converted to numbers using the Number() function.

When used as a function, Number(value) converts a string or other value to the Number type. If the value can't be converted, it returns NaN (The NaN global property is a value representing Not-A-Number.)

For example

```
let score="33"
console.log(typeof(score))
let valueInNumber=Number(score)
console.log(typeof(valueInNumber))
let num="3A"
let numInNumber=Number(num)
console.log(numInNumber)
```

The Boolean() Function

We can use the Boolean() function to find out if an expression (or a variable) is true:

Everything With a "Value" is True

Examples

100

3.14

-15

"Hello"

"false"

7 + 1 + 3.14

Everything Without a "Value" is False

The Boolean value of 0 (zero) is false:

let x = 0;

Boolean(x);

The Boolean value of -0 (minus zero) is false:

let x = -0;

Boolean(x);

The Boolean value of "" (empty string) is false:

let x = "";

Boolean(x);

The Boolean value of undefined is false:

let x;

Boolean(x);

The Boolean value of null is false:

let x = null;

Boolean(x);

The Boolean value of false is (you guessed it) false:

let x = false;

Boolean(x);

The Boolean value of NaN is false:

let x = 10 / "Hallo";

Boolean(x);

JavaScript switch Statement

The JavaScript `switch` statement is used in decision making.

The `switch` statement evaluates an expression and executes the corresponding body that matches the expression's result.

The syntax of the `switch` statement is:

```
switch(expression) {  
  case value1:  
    // body of case 1
```

```
break;

case value2:
    // body of case 2
    break;

case valueN:
    // body of case N
    break;

default:
    // body of default
}
```

The `switch` statement evaluates a variable/expression inside parentheses `()`.

- If the result of the expression is equal to `value1`, its body is executed.
- If the result of the expression is equal to `value2`, its body is executed.
- This process goes on. If there is no matching case, the `default` body executes.

Notes:

- The `break` statement is optional. If the `break` statement is encountered, the `switch` statement ends.
- If the `break` statement is not used, the cases after the matching case are also executed.
- The `default` clause is also optional.

Example 1: Simple Program Using switch Statement

```
// program using switch statement
let a = 2;
```

```
switch (a) {

    case 1:
        a = 'one';
        break;
    case 2:
        a = 'two';
        break;
    default:
        a = 'not found';
        break;
}
```

```
}  
document.write(`The value is ${a}`);
```

Example 2: Type Checking in switch Statement

```
// program using switch statement  
let a = 1;
```

```
switch (a) {  
  case "1":  
    a = 1;  
    break;  
  case 1:  
    a = 'one';  
    break;  
  case 2:  
    a = 'two';  
    break;  
  
  default:  
    a = 'not found';  
    break;  
}  
document.write(`The value is ${a}`);
```

In the above program, an expression `a = 1` is evaluated with a `switch` statement.

- In JavaScript, the switch statement checks the value strictly. So the expression's result does not match with `case "1"`.
- Then the `switch` statement goes to the second case. Here, the expression's result matches with `case 1`. So `The value is one` is displayed.
- The `break` statement terminates the block and control flow of the program jumps to outside of the `switch` block.

Note: In JavaScript, the switch statement checks the cases strictly (should be of the same data type) with the expression's result. Notice in the above example, `1` does not match with `"1"`.

Let's write a program to make a simple calculator with the `switch` statement.

Example 3: Simple Calculator

```
// program for a simple calculator
```

```
let result;
```

```
// take the operator input
```

```
let operator = prompt('Enter operator ( either +, -, * or / ): ');
```

```
// take the operand input
```

```
let number1 = parseInt(prompt('Enter first number: '));
```

```
let number2 = parseInt(prompt('Enter second number: '));
```

```
switch(operator) {
```

```
  case '+':
```

```
    result = number1 + number2;
```

```
    document.write(`${number1} + ${number2} = ${result}`);
```

```
    break;
```

```
  case '-':
```

```
    result = number1 - number2;
```

```
    document.write(`${number1} - ${number2} = ${result}`);
```

```
    break;
```

```
  case '*':
```

```
    result = number1 * number2;
```

```
    document.write(`${number1} * ${number2} = ${result}`);
```

```
    break;
```

```
  case '/':
```

```
    result = number1 / number2;
```

```
    document.write(`${number1} / ${number2} = ${result}`);
```

```
    break;
```

```
  default:
```

```
    document.write('Invalid operator');
```

```
    break;
```

```
}
```

JavaScript switch With Multiple Case

In a JavaScript switch statement, cases can be grouped to share the same code.

Example 4: switch With Multiple Case

```
// multiple case switch program
let fruit = 'apple';
switch(fruit) {
  case 'apple':
  case 'mango':
  case 'pineapple':
    document.write(`${fruit} is a fruit.`);
    break;
  default:
    document.write(`${fruit} is not a fruit.`);
    break;
}
```

JavaScript History

- JavaScript was invented by Brendan Eich in 1995, and became an ECMA standard in 1997.
- ECMAScript is the official name of the language.
- ECMAScript versions have been abbreviated to ES1, ES2, ES3, ES5, and ES6.
- Since 2016, versions are named by year (ECMAScript 2016, 2017, 2018, 2019, 2020).

ECMAScript 2023, the 14th and current version, was released in June 2023.

JavaScript ES6

JavaScript **ES6** (also known as **ECMAScript 2015** or **ECMAScript 6**) is the newer version of JavaScript that was introduced in 2015. ECMAScript is the standard that JavaScript programming language uses. ECMAScript provides the specification on how JavaScript programming language should work.

JavaScript Function and Function Expressions

A JavaScript function is a block of code designed to perform a particular task. A JavaScript function is executed when "something" invokes it (calls it). It helps to reuse the code

A JavaScript function is defined with the **function** keyword, followed by a **name**, followed by parentheses **()**.

Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).

The code to be executed, by the function, is placed inside curly brackets: {}

```
function functionname() {  
    // code to be executed  
}
```

For example

//Defining function

```
let a=10;
```

```
let b=20;
```

//calling a function

```
sum(a,b);
```

```
sum(1,2);
```

//function definition

```
function sum(a,b){
```

```
    let s=a+b;
```

```
    console.log(s);
```

```
}
```

//Defining function with return statement

```
let a=10;
```

```
let b=20;
```

//calling a function

```
let t=sum(a,b); //argument
```

```
console.log(t);
```

```
t=sum(1,2);
```

```
console.log(t);
```

//function definition with return statement

//they are called parameter

//function declaration

```
function sum(a,b){
```

```
    let s=a+b;
```

```
    return s;
```

```
}
```

Declaring a Function

The syntax to declare a function is:

```
function nameOfFunction () {
```

```
    // function body
```

```
}
```

A function is declared using the function keyword.

The basic rules of naming a function are similar to naming a variable. It is better to write a descriptive name for your function. For example, if a function is used to add two numbers, you could name the function add or addNumbers.

The body of function is written within {}.

For example,

```
// declaring a function named greet()
function greet() {
  console.log("Hello there");
}
```

Calling a Function

In the above program, we have declared a function named greet(). To use that function, we need to call it.

Here's how we can call the above greet() function.

```
// function call
greet();
```

Example : Display a Text

```
// program to print a text
// declaring a function
function greet() {
  console.log("Hello there!");
}
// calling the function

greet();
```

Function Parameters

A function can also be declared with parameters. A parameter is a value that is passed when declaring a function.

```
// program to print the text
// declaring a function
function greet(name) {
  console.log("Hello " + name + ":");
}
// variable name can be different
```

```
let name = prompt("Enter a name: ");  
// calling function  
greet(name);
```

Note: When a value is passed when declaring a function, it is called parameter. And when the function is called, the value passed is called argument.

Example: Add Two Numbers

```
// program to add two numbers using a function  
// declaring a function  
function add(a, b) {  
    console.log(a + b);  
}  
// calling functions  
add(3,4);  
add(2,9);
```

Notice that we can call a function as many times as you want. We can write one function and then call it multiple times with different arguments.

Function Return

The return statement can be used to return the value to a function call. The return statement denotes that the function has ended. Any code after return is not executed. If nothing is returned, the function returns an undefined value.

Example: Sum of Two Numbers

```
// program to add two numbers  
// declaring a function  
function add(a, b) {  
    return a + b;  
}  
// take input from the user  
let number1 = parseFloat(prompt("Enter first number: "));  
let number2 = parseFloat(prompt("Enter second number: "));  
// calling function  
let result = add(number1,number2);  
// display the result  
console.log("The sum is " + result);
```

Default Parameters

The concept of default parameters is a new feature introduced in the **ES6 (ECMAScript 2015 allows default parameter values in the function declaration)** version of JavaScript. This allows us to give default values to function parameters. Let's take an example,

```
function sum(x = 3, y = 5) {  
  // return sum  
  return x + y;  
}  
console.log(sum(5, 15)); // 20  
console.log(sum(7));     // 12  
console.log(sum());      // 8
```

If a function in JavaScript is called with missing arguments (less than declared), the missing values are set to undefined.

Sometimes this is acceptable, but sometimes it is better to assign a default value to the parameter:

```
function myFunction(x, y) {  
  if (y === undefined) {  
    y = 2;  
  }  
  return x * y;  
}
```

JavaScript undefined

The undefined is a primitive type in JavaScript. So the undefined is a type. And the undefined type has exactly one value that is undefined.

JavaScript uses the undefined value in the following situations:

Accessing an uninitialized variable

When we declare a variable and don't initialize it to a value, the variable will have a value of undefined. For example:

```
let counter;  
console.log(counter); // undefined
```

It's a good practice to always initialize a variable whenever possible. In this example, we can initialize the counter variable to zero, like this:

```
let counter = 0;  
console.log(counter); // 0
```

Setting JavaScript default parameters for a function

In JavaScript, a parameter has a default value of undefined. It means that if you don't pass the arguments into the function, its parameters will have the default values of undefined.

See the following example:

```
function say(message) {  
    console.log(message);  
}  
say(); // undefined
```

The say() function takes the message parameter. Because we didn't pass any argument into the say() function, the value of the message parameter is undefined.

Suppose that you want to give the message parameter a default value 10.

A typical way for achieving this is to test parameter value and assign a default value if it is undefined using a ternary operator:

```
function say(message) {  
    message = typeof message !== 'undefined' ? message : 'Hi';  
    console.log(message);  
}  
say(); // 'Hi'
```

In this example, we didn't pass any value into the say() function. Therefore, the default value of the message argument is undefined

Using Expressions as Default Values

It is also possible to provide expressions as default values.

Example : Passing Parameter as Default Values

```
function sum(x = 1, y = x, z = x + y) {  
    console.log(x + y + z);  
}
```

If we reference the parameter that has not been initialized yet, we will get an error. For example,

```
function sum(x = y, y = 1) {  
    console.log(x + y);  
}  
sum();
```

Passing undefined Value

In JavaScript, when we pass undefined to a default parameter function, the function takes the default value. For example,

```
function test(x = 1) {  
    console.log(x);  
}
```

```
// passing undefined  
// takes default value 1  
test(undefined); // 1
```

Benefits of Using a Function

- Function makes the code reusable. We can declare it once and use it multiple times.
- Function makes the program easier as each small task is divided into a function.
- Function increases readability.

Function Expression

The Javascript Function Expression is used to define a function inside any expression. The Function Expression allows us to create an anonymous function that doesn't have any function name which is the main difference between Function Expression and Function Declaration. In function Expression function is treat as a value.

```
let variableName = function(x, y) { statements... return (z) };
```

First-Class Function language: A programming language is said to have First-class functions if functions in that language are treated like other variables. So the functions can be assigned to any other variable or passed as an argument or can be returned by another function.

For example

```
//Function Expression  
let avg=function(a,b){  
    return (a+b)/2;  
}  
//now invoke the function  
let r=avg(4,5);  
console.log(r);
```

Whenever we define function expression , we can call it after defining it (other wise it produce error)

```
let calSub = function (x, y) {  
    let z = x - y;  
    return z;  
}
```

```
console.log("Subtraction : " + calSub(7, 4));
```

Array

Array is a single Object that can hold multiple values

Collection of items(could be of same type or different type)

Array contain length Property-which return number of items in any array

In javascript Array are dynamic there size are not fixed

How to define an array

```
let arr=[];
let numArray=[10,20,30];
let anotherArray=[];
anotherArray[0]="Monday";
anotherArray[1]="Tuesday";
//program1
let nums=[1,2,3];
console.log(nums);
console.log(nums.length)
//program2
let greetings=["Hi","HELLO","NAMASTE"];
console.log(greetings);
greetings[3]="Message";
greetings[4]="ABC";
console.log(greetings);
//To access element of an array
console.log(greetings[0]);
console.log(greetings[1]);
console.log(greetings[2]);
//if we want to access any element which is not in range
console.log(greetings[20]); //return undefined
//program3
let cars = new Array("Saab", "Volvo", "BMW");
console.log(cars);
//program
let fruits=["Apple","Mango","Kiwi"];
console.log(fruits);
//The push() method adds an element at the end of the array.
fruits.push('Banana');
console.log(fruits);
//We can add multiple value with push() method
```

```
fruits.push('c','d');
console.log(fruits);
//The pop() method to remove the last element from an array.
// The pop() method also returns the value
let fruit=fruits.pop();
console.log(fruit)
console.log(fruits)
//If we need to remove the first element, we can use the shift() method. The
shift() method //removes the first element and also returns the removed
element.
fruit=fruits.shift();
console.log(fruit)
console.log(fruits)
//The unshift() method adds an element at the beginning of the array
fruits.unshift('Orange');
console.log(fruits);
//We can add multiple values with unshift() method
fruits.unshift('Apricot','Papaya');
console.log(fruits);
//Program
let fruits=["Apple","Mango","Kiwi"];
console.log(fruits);
//indexOf() searches an element of an array and returns its index ,if not found
then return -1
console.log(fruits.indexOf('Papaya'))
console.log(fruits.indexOf('Peach'))
// The join() method returns an array as a string.
// The join() method does not change the original array.
// Any separator can be specified. The default is comma (,).
console.log(fruits.join());
console.log(fruits.join('-'));
```

Splice Method

The splice() method adds and/or removes array elements.
The splice() method overwrites the original array.

```
//Program
let fruits= ["Banana", "Orange", "Apple", "Mango"];
console.log(fruits);
//remove elements through splice
//splice from position 1 delete 2 element
fruits.splice(1,2);
```

```
console.log(fruits);  
//add two element from index 1  
//second parameter is zero because we dont want to delete any element  
fruits.splice(1,0,'splice1','splice2');  
console.log(fruits);  
//add one element from index 1 and remove one element from index 1 (it is  
sort of replace)  
fruits.splice(1,1,'splice3');  
console.log(fruits);
```

Array Const

- The property of a const object can be changed but it cannot be changed to a reference to the new object
- The values inside the const array can be changed, it can add new items to const arrays but it cannot reference a new array.
- Re-declaring of a const variable inside different block scopes is allowed.

Cannot be Reassigned

An array declared with const cannot be reassigned:

Elements Can be Reassigned

We can change the elements of a constant array:

```
// Changing the content of array is  
// possible in cost array  
const names = ["pankaj", "sumit", "chandan", "ajay"];  
console.log(names);  
names[2] = "Narayan"; // possible  
console.log(names);  
names.push("ankit"); //possible  
console.log(names);
```

Assigned when Declared

JavaScript const variables must be assigned a value when they are declared:

Meaning: An array declared with const must be initialized when it is declared.

Using const without initializing the array is a syntax error:

This will not work:

```
const cars;  
cars = ["Saab", "Volvo", "BMW"];
```

Const Block Scope

An array declared with const has **Block Scope**.

An array declared in a block is not the same as an array declared outside the block:

```
const cars = ["Saab", "Volvo", "BMW"];
// Here cars[0] is "Saab"
console.log(cars[0])
{
  const cars = ["Toyota", "Volvo", "BMW"];
  // Here cars[0] is "Toyota"
  console.log(cars[0])
}
// Here cars[0] is "Saab"
console.log(cars[0])
//Program of Iteration
let fruits= ["Banana", "Orange", "Apple", "Mango"];
for(let i=0;i<fruits.length;i++)
{
  console.log(fruits[i]);
}
```

Looping Array Elements –

The safest way to loop through an array, is using a “for” loop:

```
let fruits, text, fLen;
fruits = ["Banana", "Orange", "Apple", "Mango"];
fLen = fruits.length;
text = "<ul>";
for (let i = 0; i < fLen; i++) {
  text += "<li>" + fruits[i] + "</li>";
}
text += "</ul>";
document.write(text);
```

For In Over Arrays

The JavaScript for in statement can also loop over the properties of an Array:
But it is not preferred for large data set

Syntax

```
for (variable in array) {  
  code  
}  
let fruits = ["Banana", "Orange", "Apple", "Mango"];
```

```
for(let index in fruits)  
{  
  console.log(fruits[index]);  
}
```

The For Of Loop

The JavaScript for of statement loops through the values of an iterable object.

It simply use value not index

```
let fruits = ["Banana", "Orange", "Apple", "Mango"];  
for(let fruit of fruits)  
{  
  console.log(fruit);  
}
```

Value & Reference Types

Value Types :- Value is literally stored in the variables

Reference Types : Address is stored in the variables

Primitive and Reference value in JavaScript

When we access a primitive value, we manipulate the actual value stored in that variable. Thus, variables that are primitive are **accessed by Value**. When we assign a variable that stores a primitive value to another, the value stored in the variable is created and copied into the new variable.

Value is literally stored in a variable

```
Let a=5;  
Let b=a;  
Let a=10;
```

When we access a reference value, we manipulate it through reference, not through its actual value that is stored. Thus, variables that are reference values are **accessed by reference**. When we assign a reference value from

one variable to another, the value stored in the variable is also copied into the location of the new variable but the difference is that the values stored in both variables now are the address of the actual object stored on the heap. As a result, both variables are referencing the same object, So we can manipulate the original object from both variables.

Address store in the variable

```
//program of swapping with function
a=[10,20]
//value of a before swapping
console.log(a)
function swap(b)
{
    let t;
    t=b[0];
    b[0]=b[1];
    b[1]=t;
}
swap(a);
//value of a after swapping
console.log(a);
```

Call by Value Vs Call by Reference

Call by Value: In this method, values of actual parameters are copied to the function's formal parameters, and the two types of parameters are stored in different memory locations. So any changes made inside functions are not reflected in the actual parameters of the caller

Call by Value:

In this method, values of actual parameters are copied to the function's formal parameters, and the two types of parameters are stored in different memory locations. So any changes made inside functions are not reflected in the actual parameters of the caller.

Suppose there is a variable named "a". Now, we store a primitive value(boolean, integer, float, etc) in the variable "a". Let us store an integer value in "a", Let a=5. Now the variable "a" stores 5 and has an address location where that primitive value sits in memory.

Now, suppose we copy the value of "a" in "b" by assignment (a=b). Now, "b" points to a new location in memory, containing the same data as variable "a". Thus, a=b=5 but both point to separate locations in memory.

This approach is called call by value where 2 variables become the same by copying the value but in 2 separate spots in the memory.

Features of call by value:

- Function arguments are always passed by value.
- It copies the value of a variable passed in a function to a local variable.
- Both these variables occupy separate locations in memory. Thus, if changes are made in a particular variable it does not affect the other one.

Call by reference: Let's say, we have an object stored in the variable "a". The variable stores the location or the address where the object lives. Now we set b=a. Now that new variable "b" instead of pointing to a new location in the memory, points to the same location where "a" does. No new object is created, and no copy is created. Both variables point to the same object. This is like having 2 names.

This is called by reference. It behaves quite differently from by value. All objects interact by reference.

Features of By reference:

- In JavaScript, all objects interact by reference.
- If an object is stored in a variable and that variable is made equal to another variable then both of them occupy the same location in memory.
- Changes in one object variable affect the other object variable.

For example

```
//program of swapping with function
a=[10,20]
//value of a before swapping
console.log(a)
function swap(b)
{
  let t;
  t=b[0];
  b[0]=b[1];
  b[1]=t;
}
swap(a);
//value of a after swapping
```

```
console.log(a);
```

High-level languages are programming languages that are designed to allow humans to write computer programs and interact with a computer system without having to have specific knowledge of the processor or hardware that the program will run on.

Low Level Language

A low-level language, often known as a computer's native language, is a sort of programming language. It is very close to writing actual machine instructions, and it deals with a computer's hardware components and constraints.

Difference between Compiler and Interpreter

The Compiler and Interpreter, both have similar works to perform. Interpreters and Compilers convert the Source Code (HLL) to Machine Code (understandable by Computer). In general, computer programs exist in High-Level Language that a human being can easily understand. But computers cannot understand the same high-level language, so for computers, we have to convert them into machine language and make them readable for computers.

Compiler

The Compiler is a translator which takes input i.e., High-Level Language, and produces an output of low-level language i.e. machine or assembly language. The work of a Compiler is to transform the codes written in the programming language into machine code (format of 0s and 1s) so that computers can understand.

Interpreter

An Interpreter is a program that translates a programming language into a comprehensible language. The interpreter converts high-level language to an intermediate language. It contains pre-compiled code, source code, etc.

Javascript Engine

JavaScript is a scripting language and is not directly understood by computer but the browsers have inbuilt JavaScript engine which help them to understand and interpret JavaScript codes. These engines help to convert our JavaScript program into computer-understandable language.

A JavaScript engine is a computer program that executes JavaScript code and converts it into computer understandable language.

Browser	Name of Javascript Engine
Google Chrome	V8
Edge (Internet Explorer)	Chakra
Mozilla Firefox	Spider Monkey
Safari	Javascript Core Webkit

JavaScript Interpreted or Compiled

In short **Just in time compiler** is nothing but a combination of an interpreter and a compiler.

In javascript program execute in two step:

In first Step it watch our program from top to bottom and check how many variables and functions are declared and allocate memory for them(it will not put value in them, only allocate the memory)

In Second Step (Execution Phase), it put value in the variable ,execute some functions and on basis of this it produce the output

For example

```
var a=20;
```

```
let c=40;
```

```
function f(){  
    console.log("Function Declaration");  
}
```

```
const gn=function(){  
    console.log("Function Expression");  
}
```

From above code first it declare variable a and c in the memory with undefined(it only allocate the memory of the variable not the value, which will be done in execution phase)

Now it store the function definition and assign to f (not execute the code) and then it allocate memory to variable gn with undefined(it assign the function at execution phase)

This is our first step it is also known as JIT compilation

Next step is called Execution Step (Interpreter)

In this step code will get executed, it will assigned value to variable a and c and assigned function to gn variable.

Hoisting

Variable and Function declarations are moved to the top of the code

For example

```
console.log(a);
```

```
var a=20;
```

```
console.log(a);
```

Now program execute in two steps

Compilation + Execution

In Compilation phase it will not execute any thing only allocate memory to the variable with undefined.

In Execution phase it check whether variable with name a exist in the memory if it is available the print the value(which is undefined) then it assigned value 20 to variable a and then print the value of a

Now take another example

```
fn();
```

```
function fn(){
```

```
  console.log("function declaration");
```

```
}
```

In Compilation phase it will not execute any thing only assign function body to fn function

In Execution phase it check whether function body exists and then run the function

Let take another example of function expression

```
gn()
```

```
var gn=function(){
```

```
  console.log("Function Expression")
```

```
}
```

In Compilation phase it will not execute any thing only allocate memory to variable `gn` with value undefined

In Execution phase it try to execute `gn` (undefined) which produce error :

Uncaught TypeError: gn is not a function

Now let take another example

```
console.log(a);  
let a=20;  
console.log(a);
```

Whenever we declare variable with `let` and `const` it allocate the memory to the variable but before initialization we cannot process it

JavaScript **Hoisting** refers to the process whereby the interpreter appears to move the *declaration* of functions, variables, classes prior to execution of the code.

With JavaScript functions, it is possible to call functions before actually writing the code for the function statement and they give a defined output. This property is called hoisting. Hoisting is the ability of a function to be invoked at the top of the script before it is declared.

Hoisting refers to the process ,Where the interpreter appears to move the declarations to the top of the code before execution(So whatever declarations are there before execution so what JavaScript interpreter does is ,it appears to be shifted at the top of the code)

Variable can thus be references before they are declared in javaScript

JavaScript only hoist declaration not initialization

```
console.log(a)  
greet()  
function greet(){  
    console.log("Good Morning");  
}  
var a=9;//declaration hoisted to the top but not the initialization  
console.log(a);
```

The let and const Keywords

Variables defined with let and const are hoisted to the top of the block, but not *initialized*.

Meaning: The block of code is aware of the variable, but it cannot be used until it has been declared.

Using a let variable before it is declared will result in a ReferenceError.

The variable is in a "temporal dead zone" from the start of the block until it is declared:

Example

This will result in a ReferenceError:

```
carName = "Volvo";  
let carName;
```

Temporal Dead Zone in JavaScript

Temporal Dead Zone is a behaviour that occurs with variables declared using let and const keywords.

It is a behaviour where we try to access a variable before it is initialized

let and const have two differences from var

They are block scoped

Accessing a var before it is declared has the result undefined; accessing a let or const before it is declared throws ReferenceError

Or we can say let and const also hoisted but they cannot be accessed before initialization

Hoisting Example

First example

```
var a=20;  
var a;  
console.log(a);  
Second Example
```

```
var a=10;  
console.log(a);
```

```
function fn(){  
    console.log(a);  
    var a=20;  
    console.log(a);  
}  
fn();
```

In this example in compilation phase variable a will allocate the memory and body of the function is assigned to fn() .

In Execution phase value a will print and fn() will execute (before the execution of function fn() the compilation phase will execute after then execution phase will execute.)

Function

Consider the following statement

```
function print(a,b)  
{  
    console.log(arguments);  
    console.log(a);  
    console.log(b);  
    return a+ " " +b;  
}  
console.log(print())
```

```
console.log(print(1))
```

```
console.log(print(1,2,3,4))
```

what will the output ?

Javascript is a dynamic language so it ignore extra argument and if we pass less arguments it replace it with undefined

Argument is used determine how many arguments are passed.

Through arguments object we can find how many arguments are passed and based on this we can print all arguments. For example

```
function sum()  
{  
    let a=arguments;  
    //console.log(a);  
    for(let i=0;i<a.length;i++)  
    {  
        console.log(a[i]);  
    }  
}
```

```
sum(1,2,3);
```

```
sum(10,20,30,40);
```

Provide default value to the function arguments

```
function printmsg(count=1)  
{  
    for(let i=0;i<count;i++)  
    {
```

```
        console.log(`Hello ${i}`)  
    }  
}
```

```
printmsg();  
printmsg(4);
```

Arrow Function

Arrow functions were introduced in ES6.

Arrow functions allow us to write shorter function syntax:

```
let myFunction = (a, b) => a * b;
```

For example:

```
function avg(a,b)  
{  
    let r=(a+b)/2;  
    return r;  
}  
  
//function expression  
const avg2=function(a,b)  
{  
    let r=(a+b)/2;  
    return r;  
}
```

```
//arrow function
const avg3=(a,b)=>
{
    let r =(a+b)/2;
    return r;
}
console.log(avg3(2,5));

//Simplify the avg function
const avg4=(a,b)=>
{
    return (a+b)/2;
    //return r;
}
console.log(avg4(2,5));

//We can remove braces and function
const avg5=(a,b)=>(a+b)/2;
console.log(avg5(2,5));
const product=function(num){
    return num*5;
}
const product1=(num)=>num*5;
console.log(product1(5));
```

//if we have single argument then we can remove parenthesis (for 0 and more than 1 argument we require parenthesis)

```
const product2=num=>num*5;
```

```
console.log(product2(7));
```

Before Arrow:

```
hello = function() {  
  return "Hello World!";  
}
```

With Arrow Function:

```
hello = () => {  
  return "Hello World!";  
}
```

It gets shorter! If the function has only one statement, and the statement returns a value, we can remove the brackets and the return keyword:

Arrow Functions Return Value by Default:

```
hello = () => "Hello World!";
```

Note: This works only if the function has only one statement.

If we have parameters, we can pass them inside the parentheses:

Arrow Function With Parameters:

```
hello = (val) => "Hello " + val;
```

In fact, if we have only one parameter, we can skip the parentheses as well:

Arrow Function Without Parentheses:

```
hello = val => "Hello " + val;
```

What is a foreach loop

A forEach loop (method) is used to get data from JS array or object

The method calls a function once for each element in an array, in order

For example:

```
let a = ["Rahul", "Karan", "Aman", "Neha"]
a.forEach(function (name) {
  console.log("Name:-" + name);
});
```

We can also include index in function

```
let a = ["Rahul", "Karan", "Aman", "Neha"]
a.forEach(function (name, index) {
  console.log(`Name ${name} is at index ${index}`);
});
```

```
let a = ["Rahul", "Karan", "Aman", "Neha"]
a.forEach(loop);
//we can call function outside
function loop(name, index){
  console.log(`Name ${name} is at index ${index}`);
}
```

Array For each with arrow function

```
let nums=[42,51,24,98,65,12];
nums.forEach((num)=>{
  console.log(num);
});
```

With single parameter in arrow function we can skip the parenthesis

```
let nums=[42,51,24,98,65,12];
nums.forEach(num=>{
  console.log(num);
});
```

We can pass three parameter with foreach (value,index and array itself)

```
let nums=[42,51,24,98,65,12];  
nums.forEach((num,index,arr)=>{  
  console.log(num,index,arr);  
});
```

```
const cars=["car1","car2","car3"]
```

```
cars.forEach(myfunction);
```

```
function myfunction(item,index,array)
```

```
{
```

```
  //body of the function
```

```
}
```

item - (here "car1") required parameter

index- (here 0 for "car1") optional

array- (cars) optional

Here name of the variable are not fixed but position are fixed

For example

```
const cars=["car1","car2","car3","car4","car5"];
```

```
cars.forEach(car);
```

```
function car(item){
```

```
  console.log(item)
```

```
}
```

With index variable

```
const cars=["car1","car2","car3","car4","car5"];

cars.forEach(car);

function car(item,index){

    console.log(item + " "+ index);

}
```

With index and array variable

```
const numbers = [65, 44, 12, 4];

console.log(numbers);

numbers.forEach(myFunction)

function myFunction(item, index, arr) {

    arr[index] = item * 10;

}

console.log(numbers);
```

Considering that we have the following array below:

```
const numbers = [1, 2, 3, 4, 5];
```

Firstly, to loop through an array by using the `forEach` method, you need a callback function (or anonymous function):

```
numbers.forEach(function() {

    // code

});
```

The function will be executed for every single element of the array. It must take at least one parameter which represents the elements of an array:

```
numbers.forEach(function(number) {  
    console.log(number);  
});
```

Alternatively, we can use the ES6 arrow function representation for simplifying the code:

```
numbers.forEach(number => console.log(number));
```

Basically, we can see the index number of an element if we include it as a second parameter:

```
numbers.forEach((number, index) => {  
    console.log('Index: ' + index + ' Value: ' + number);  
});
```

Array

The array parameter is the array itself. It is also optional and can be used if necessary in various operations. Otherwise, if we call it, it will just get printed as many times as the number of elements of the array:

```
numbers.forEach((number, index, array) => {  
    console.log(array);  
});
```

Now full program will be like this :

```
const numbers = [1, 2, 3, 4, 5];

//use anonymous function
numbers.forEach(function(number) {
    console.log(number);
});

//use arrow function with single parameter
numbers.forEach(number => console.log(number));

//use arrow function with two argument
numbers.forEach((number, index) => {
    console.log('Index: ' + index + ' Value: ' + number);
});

console.log(numbers);

numbers.forEach((number, index, array) => {
    array[index]=number*10;
});

console.log(numbers);
```

Array.from()

The Array.from() method returns an array from any object

```
let text = "ABCDEFGG"
const myArr = Array.from(text);
console.log(myArr)
```

The For Of Loop

The JavaScript for of statement loops through the values of an iterable object.

```
const cars = ["BMW", "Volvo", "Mini"];
let text = "";
for (let x of cars) {
  console.log(x);
}
```

For In Loop

for in statement can also loop over the properties of an Array

```
let myString="This is the string";
for(let index in myString)
{
  console.log(myString[index]);
}
```

```
const numbers = [45, 4, 9, 16, 25];
for (let x in numbers) {
  console.log(numbers[x]);
}
```

object

Object is a collection of key-Value Pairs

//let defines object

```
let student1={
  firstName:"Rakesh",
  lastName:"Sharma",
  age:22
};
console.log(student1);
let student2={
  firstName:"Amit",
```

```
lastName:"Mathur",
age:25
};
console.log(student2);
//To access first name of the student1
console.log(student1.firstName)
//or
console.log(student1["firstName"]);
//To access wrong property it will give us undefined
console.log(student1.age1)
//To add property at run time
student1.address="Aliganj";
console.log(student1);
//delete any property at run time
delete student1.address;
console.log(student1);
//To add space property in the object we use square bracket (here we can't
use dot operator)

student1["street address"]="Aliganj"
//To access space property we use square bracket (here we can't use dot
operator)
console.log(student1["street address"]);

//To access object property we can either use dot operator or square bracket
```

Nested Object

Objects that contain objects inside them are called nested or cascaded Objects. Accessing the value in nested objects can be done by consecutively applying dot or square bracket notation. Whenever we see braces inside the object it is considered to be object(nested)

For example

```
//we can also define key as string
let student1={
  firstName:"Rakesh",
  lastName:"Sharma",
  age:22,
```

```
"pin":226020,  
address:{  
  city:"Lucknow",  
  state:"UP"  
}  
};  
console.log(student1);  
console.log(student1.pin);  
  
//To access state property of address object in student1  
console.log(student1.address.state);  
//To access state property using square bracket  
console.log(student1["address"]["state"])
```

JSON.stringify()

JSON stands for JavaScript Object Notation

A common use of JSON is to exchange data to/from a web server.

When sending data to a web server, the data has to be a string.

Convert a JavaScript object into a string with JSON.stringify()

```
const obj = {name: "Rakesh", age: 30, city: "New York"};
```

Use the JavaScript function JSON.stringify() to convert it into a string.

```
const myJSON = JSON.stringify(obj);
```

Object Oriented Programming

Paradigm which allows us to solve problems with the help of objects

which represents the real world entities.

Objects contains:

Properties or Fields (Object Knows)

Method or Functions(Object Does)

Methods in Objects

For example in student object

student properties:-

firstName

lastName

age

student Methods

getFullName()

getAge()

For example object **car**

car properties

name

type(Petrol/Diesel)

fuel

car method

applybrakes()

drive()

Different way of defining Object Methods

```
let person={  
  firstName:"Amit",
```

```
    lastName: "Kumar",
    greet: function(){
        console.log("Greeting");
    },
    greet2(){
        console.log("greeting 2")
    }
};

person.age=25;

person.sayHello=function(){
    console.log("Hello!");
}

//Now declare another function with function declaration

function greet(){
    console.log("Greetings");
}
//Assigning normal function to object method
// person.greet=greet;

// person.greet();

person.sayHello();
person.greet();
person.greet2();
```

For example

```
const student1 = {
    firstName: "Rajesh",
    lastName: "Gupta",
    age: 25,
    //anonymous Functions
    getFullName: function() {
        console.log("This is the full name");
        console.log(this.firstName+" "+this.lastName);
        //this means current object
    }
};

const student2 = {
    firstName: "Amit",
```

```
lastName: "Singh",
age:35,
//anonymous Functions
getFullName: function() {
  console.log("This is the full name");
  console.log(this.firstName+" "+this.lastName);
  //this means current object
}
};
student1.getFullName();//when we call getFullName from student1 so student1 is the
current object
student2.getFullName();//when we call getFullName from student2 so student2 is the
current object
```

why we need to use the this keyword

If we just use firstName and lastName, it will be looking for a global variable (ie, a variable defined outside of the object). We need to use this.someVariable so the computer knows we're talking about a property of the object, not a global variable.

```
//These won't be used, because the person.sayHello method will use person.name
and person.age as defined below
var name = "Vipul Jain";
var age = 57;

var person = {
  name: "Rakesh Mehra",
  age: 30,
  sayHello: function () {
    console.log("Hi, my name is " + this.name + " and I am " + this.age +
" years old"); //uses this.property
  }
};

//Now it will print person's name/age, and not Vipul's
person.sayHello();
```

Another example

```
let car1={
  name:"Honda",
  type:"Petrol",
  fuel:10,
  drive:function(){
```

```
if(this.fuel>0)
  console.log("Car is Driving");
else
  console.log("Please refill");
}
};

car1.drive();
car1.fuel=0;//here we are changing state of the object
car1.drive(); //The behaviour is change automatically
```

Wrapper Objects & Built-In JavaScript Object

Wrapper are of three types

Number
String
Boolean

Behind the scene Whenever we access these values they are treated as object and after completing the task it remove the object (treated as value)

For example

```
let msg="Hello";
let number=20;
```

//Like object we can assign property to the string for example

```
msg.type="Message";//here it create new object assign the value and discard it
```

```
console.log(msg.type); // It will display undefine because now msg does not have any property
```

//with name type,in this line also new object created

JavaScript provides three primitive wrapper types: Boolean, Number, and String types.

The primitive wrapper types make it easier to use primitive values including booleans, numbers, and strings.

See the following example:

```
let language = 'JavaScript';  
let s = language.substring(4);  
console.log(s); // Script
```

In this example, The variable language holds a primitive string value. It doesn't have any method like substring(). However, the above code works perfectly. When you call a method on a variable that holds a number, a string, or a boolean, JavaScript performs the following steps behind the scenes:

- Create an object of a corresponding type.
- Call a specific method on the instance.
- Delete the instance immediately.

Math Object

The JavaScript Math object allows you to perform mathematical tasks on numbers.

```
console.log(Math.PI)  
console.log(Math.floor(4.5))//4  
console.log(Math.ceil(4.5))//larger value 5  
console.log(Math.round(4.3))//return 4  
console.log(Math.round(4.7))//return 5  
console.log(Math.random())//return value between 0 and 1  
console.log(Math.random())//return value between 0 and 1  
console.log(Math.random())//return value between 0 and 1  
let pizza=["Pizza 1","Pizza 2","Pizza 3"];  
//index value 0,1,2  
let num=Math.floor(Math.random()*3)  
console.log(pizza[num])  
num=Math.floor(Math.random()*3)  
console.log(pizza[num])  
num=Math.floor(Math.random()*3)  
console.log(pizza[num])
```

JavaScript Strings

JavaScript strings are for storing and manipulating text.

A JavaScript string is zero or more characters written inside quotes.

We can use single or double quotes:

Example

```
let carName1 = "Volvo XC60"; // Double quotes  
let carName2 = 'Volvo XC60'; // Single quotes  
Character in a string are indexed starting from 0  
length Property - number of characters in string  
let msg="Computer";  
console.log(msg.length)  
console.log(msg[2])
```

Escape Character

Because strings must be written within quotes, JavaScript will misunderstand this string:

```
let text = "We are the so-called "Vikings" from the north.";
```

The string will be chopped to "We are the so-called ".

The solution to avoid this problem, is to use the backslash escape character.

The backslash (\) escape character turns special characters into string characters:

Code	Result	Description
\'	'	Single quote
\"	"	Double quote
\\	\	Backslash
\n		New line
\t		Tab

The sequence \" inserts a double quote in a string:

Example

```
let text = "We are the so-called \"Ramesh\" from the north.";
```

```
for(let i=0;i<msg.length; i++)
{
    console.log(msg[i]);
}
//for of loop
for (const char of msg) {
    console.log(char);
};
//for each loop
Array.from(msg).forEach(elem => {
    console.log(elem);
});
//for in loop
msg="Rakesh";
for (let x in msg){
    console.log(msg[x])
}
```

String Interpolation

String Interpolation is also known as template string or template literal. String interpolation is a great programming language feature that allows injecting variables, function calls, arithmetic expressions directly into a string. String interpolation was absent in JavaScript before ES6. String interpolation is a new feature of ES6, that can make multi-line strings without the need for an escape character. We can use apostrophes and quotes easily that they can make our strings and therefore our code easier to read as well. These are some of the reasons to use string interpolation over string concatenation. Let's see the difference between string concatenation and string interpolation.

```
<script>
// String Concatenation
function myInfo(fname, lname, country) {
    return "My name is " + fname + " " + lname + ". "
        + country + " is my favorite country.";
}
console.log(myInfo("Rakesh", "Sharma", "India"));
</script>
```

In string concatenation, it is hard to maintain string as they grow large it becomes tedious and complex. In order to make it readable, the developer has to maintain all the whitespaces. This is where ES6 comes to rescue with String interpolation. In JavaScript, the template literals (strings wrapped in backticks ``) and `${expression}` as placeholders perform the string interpolation. Now we can write above myInfo function with string interpolation.

```
<script>

// String Interpolation
function myInfo(fname, lname, country) {
    return `My name is ${fname} ${lname}. ${country} is my favorite country`;
}
console.log(myInfo("Rakesh", "Sharma", "India"));
</script>
```

```
<script>
// Function call
function greet() {
    return "hello!";
}
// hello! I am a student.
console.log(`${greet()} I am a student.`);
// Expression evolution
//sum of 5 and 6 is 11.
console.log(`sum of 5 and 6 is ${5+6}.`)
</script>

<script>

    function isEven(x) {
        console.log(`x is ${x % 2 === 0 ? 'even' : 'odd'}`);
    }
    isEven(4); // x is even

</script>
```

String Immutability

Immutable means- once defined, we can't change it.

for example

```
let msg="Hello";
msg[0]="C";//won't happen
console.log(msg); //display hello
```

String Methods

toUpperCase()

toLowerCase()

indexOf()

charAt()

slice()

indexOf()

The indexOf() method returns the position of the first occurrence of a value in a string.

The indexOf() method returns -1 if the value is not found.

The indexOf() method is case sensitive i.e. if we want to search “world” it will search “world” with same case

For example

```
let msg="Hello";
let strName="AMIT";
console.log(msg.toUpperCase()); //display HELLO
console.log(strName.toLowerCase()); //display amit
let str="Hello World"
console.log(str.indexOf("World")) //display poosition 6
console.log(str.indexOf("index")) //display -1 as text is not found (indexOf is case sensitive)
console.log(str.indexOf("world")); //(indexOf is case sensitive)
```

Slice()

The `slice()` method extracts and returns a section of a string.

Example

```
const message = "JavaScript is fun";
// slice the substring from index 0 to 10
let result = message.slice(0, 10);
console.log(result);
// Output: JavaScript
```

slice() Syntax

The syntax of the slice() method is:

```
str.slice(beginIndex, endIndex)
```

Here, str is a string.

slice() Parameters

The slice() method takes in:

beginIndex - Starting index of the selection

endIndex (optional) - Ending index of the selection (Exclusive) By default, it extracts till the end of the string.

Using slice() method

```
const str = "JavaScript is a dynamic programming language.";
// from index 28 to end
console.log(str.slice(28));
// from index 4 to 14
console.log(str.slice(4, 15)); // 'Script is a'
```

substring()

The substring() method returns a specified part of the string between start and end indexes.

Example

```
const message = "JavaScript is fun.";
// get the substring starting from index 0 to 10
```

```
let result = message.substring(0, 10);  
console.log(result);  
// Output: JavaScript
```

substring() Syntax

The syntax of the substring() method is:

```
str.substring(indexStart, indexEnd)
```

Here, str is a string.

substring() Parameters

The substring() method takes in:

indexStart - The index of the first character to start including in the returned substring.

indexEnd (optional) - The index before which to stop extraction. (Exclusive) If omitted, it extracts till the end of the string.

Notes:

Any argument value < 0 is treated as 0.

Any argument value > str.length is treated as str.length.

Any NaN argument value is treated as 0.

If indexStart is greater than indexEnd, the two arguments are swapped, i.e. str.substring(a, b) will be str.substring(b, a).

substring() Return Value

Returns a new string containing the specified part of the given string.

Note: substring() does not change the original string.

Example 1: Using substring

```
let string = "Javascript is a dynamic language";  
// first character  
substr1 = string.substring(0, 1);  
console.log(substr1);  
// if start > end, they are swapped  
substr2 = string.substring(1, 0);  
console.log(substr2);  
// From 11th to last character  
substr3 = string.substring(10);  
console.log(substr3);  
// same as string.substring(0)  
substr4 = string.substring(-44, 90);  
console.log(substr4);  
// index End is exclusive  
substr5 = string.substring(0, string.length - 1);  
console.log(substr5);
```

Differences between substring() and slice()

The substring() and slice() methods are almost identical, but there are a couple of subtle differences between the two, especially in the way negative arguments are dealt with.

The substring() method swaps its two arguments if index Start is greater than index End, meaning that a string is still returned. The slice() method returns an empty string if this is the case.

```
const text = "Mozilla";  
console.log(text.substring(5, 2)); // "zil"  
console.log(text.slice(5, 2)); // ""
```

Nan in JavaScript

Nan is a JavaScript reserved word indicating that a number is not a legal number

Trying to do arithmetic with a non-numeric string will result in NaN(not a number):

For example

```
let x=100/"Apple";  
console.log(x);//output will be Nan (Not a number)
```

isNaN()

The `isNaN()` function checks if a value is **NaN (Not-a-Number)** or not.

```
let number = NaN;  
// check if number is NaN  
let result = isNaN(number);  
console.log("Is number a NaN?", result);  
// Output: Is number a NaN? True
```

JavaScript Object Literal

JavaScript Object Literal is a data type used to define objects in JavaScript. It is a syntax for creating an object in JavaScript that is composed of key-value pairs. It is a lightweight and efficient way to create and store data. It is a great way to store related data in an organized manner.

Why should you use it?

Object literals are easy to read and understand.

Object literals are more efficient than other data types.

Object literals are versatile and can store any type of data.

Object Literal

Object literals are one of the most commonly used data structures in JavaScript. They are used to store collections of data, and can be used to

represent complex data structures such as objects, arrays, functions, and even regular expressions. Object literals are also used to store information about a particular instance of an object, such as its state or behavior.

Object literals are written in the form of key-value pairs, where each key is a string and each value can be any valid JavaScript data type. For example, the following object literal stores information about a person:

```
const person = {  
  name: 'John Doe',  
  age: 30,  
  address: '123 Main Street'  
};
```

Object literals can also contain other object literals, as well as functions. For example, the following object literal stores information about a person, and also contains a function that prints out a greeting message:

```
const person = {  
  name: 'John Doe',  
  age: 30,  
  address: '123 Main Street',  
  greet: function() {  
    console.log('Hello, my name is ' + this.name);  
  }  
};
```

JavaScript Constructor Function

Let consider an example

```
let student1={  
  firstName:"Rakesh",  
  lastName:"Singh",  
  getFullName:function(){  
    console.log(`${this.firstName} ${this.lastName}`);  
  }  
}  
console.log(student1);  
//If we want to create another object we can copy the student1 object and create  
//student2 object  
let student2={  
  firstName:"Vivek",  
  lastName:"Kumar",  
  getFullName:function(){  
    console.log(`${this.firstName} ${this.lastName}`);  
  }  
}
```

```
}
console.log(student2);
//Now if we want to make 50 students object then it is not feasible to copy and paste 50
times
//To resolve this we will create one function which will return object
//We create function with function name createStudent
function createStudent(firstName,lastName)
{
    let student={
        firstName:firstName,
        lastName:lastName,
        getFullName:function(){
            console.log(`${this.firstName} ${this.lastName}`);
        }
    }
    return student;
}
let student3=createStudent("Rajiv","Gupta");
console.log(student3);

//Now we create student object through constructor function
//We use this keyword to represent properties of object

function Student(firstName,lastName)
{
    //it will create new object with this
    //this={} empty object
    // now add property through this keyword
    this.lastName=lastName;
    this.getFullName=function(){
        console.log(`${this.firstName} ${this.lastName}`);
    }
    //at last it return the object
}
//To create object we have to new keyword with constructor function
//for example
let student4=new Student("Mayur","Singh");
//when we use new keyword it create an empty object in which properties are add through
this keyword
console.log(student4);
```

In JavaScript, a constructor function is used to create objects. For example,

// constructor function

```
function Person () {
    this.name = 'Rakesh',
    this.age = 23
```

```
}  
// create an object  
const person = new Person();
```

To create an object from a constructor function, we use the new keyword.

Note: It is considered a good practice to capitalize the first letter of your constructor function.

Create Multiple Objects with Constructor Function

In JavaScript, we can create multiple objects from a constructor function. For example,

```
// constructor function  
function Person () {  
    this.name = 'Rakesh',  
    this.age = 23,  
  
    this.greet = function () {  
        console.log('hello');  
    }  
}
```

```
// create objects  
const person1 = new Person();  
const person2 = new Person();
```

```
// access properties  
console.log(person1.name); // Rakesh  
console.log(person2.name); // Rakesh
```

JavaScript Constructor Function Parameters

We can also create a constructor function with parameters. For example,

```
// constructor function  
function Person (person_name, person_age, person_gender) {  
    // assigning parameter values to the calling object  
    this.name = person_name,  
    this.age = person_age,  
    this.gender = person_gender,  
  
    this.greet = function () {  
        return ('Hi' + ' ' + this.name);  
    }  
}
```

```
    }  
  }  
  // creating objects  
  const person1 = new Person('Vimal', 23, 'male');  
  const person2 = new Person('Karan', 25, 'female');  
  // accessing properties  
  console.log(person1.name); // "Vimal"  
  console.log(person2.name); // "Karan"
```

This Keyword

this keyword in JavaScript is used to refer the object it belongs to.

In a method, this refer to the owner object.

Alone, this refer to the global object

In a function, this refer to the global object

In an event, this refers to the element that received the event

Every time when we call a function Javascript set the this keyword

```
let obj={  
  firstName:"Rakesh",  
  lastName:"Singh",  
  getFullName:function(){  
    console.log(this); //it correspond to current object  
    console.log(`${this.firstName} ${this.lastName}`);  
  }  
}  
  
let obj1={  
  firstName:"Amit",  
  lastName:"Gupta",  
  getFullName:function(){  
    console.log(this);  
    console.log(`${this.firstName} ${this.lastName}`);  
  },  
  address: {  
    street: "Aliganj, Lucknow",  
    state:"Uttar Pradesh",  
    getAddress:function()  
    {  
      console.log(this);  
      console.log(this.street+" "+this.state);  
      console.log(this.firstName)// it display undefine because this correspond to address  
    }  
  }  
}  
  
obj.getFullName(); //here this correspond to current object in this case it is obj  
obj1.getFullName();//here this correspond to obj1  
obj1.address.getAddress(); //here this correspond to address
```

//this is a dynamic property , which on runtime decide to whom it belong to

Global Objects

The global object in Javascript contains variables and functions that are accessible from anywhere. These are incorporated into the language or environment by default.

The global object is known as the window object in a web browser; however, in Node.js, it is known as global.

Another program

1. this Inside Global Scope

When this is used alone, this refers to the global object (window object in browsers). For example,

```
//Alone, this refer to the global object
let a = this;
console.log(a); // Window {}
document.write("<h1>" + a + "</h1>")
this.name = 'Sarah';
console.log(window.name); // Sarah
```

Here, this.name is the same as window.name.

When this is used in a function, this refers to the global object (window object in browsers). For example,

```
function greet() {
  // this inside function
  // this refers to the global object
  console.log(this);
  document.write(this);
}

greet(); // Window {}
```

Let consider another example

```
function Person(firstName,lastName)
{
  this.firstName=firstName;
  this.lastName=lastName;
  this.getFullName=function()
  {
    console.log(this.firstName+" "+this.lastName);
  }
}
```

```
}

Person("Meher","Gautam");//here neither we use new keyword or assign to any object
//now value of this will be global object so we can call firstName,lastName and function
through window object
//as it add firstName,lastName and function to window object

console.log(window.firstName);
console.log(window.lastName);
console.log(window.getFullName());
```

call() Method

The call() method is a predefined JavaScript method.

It can be used to invoke (call) a method with an owner object as an argument (parameter).

With call(), an object can use a method belonging to another object.

```
//let consider a student constructor
function Student(firstName,lastName)
{
    this.firstName=firstName;
    this.lastName=lastName;
    this.greet=function()
    {
        console.log(` ${this.firstName} ${this.lastName}`);
    }
}
let student1=new Student("Amit","Rathi");
student1.greet();
//let consider another constructor employee with same properties
//let consider a student constructor
function Employee(firstName,lastName)
{
    this.firstName=firstName;
    this.lastName=lastName;
    this.greet=function()
    {
        console.log(` ${this.firstName} ${this.lastName}`);
    }
}
let emp=new Employee("Ravi","Verma");
emp.greet();
//Now in both constructor we have one common method name greet, which are doing
same task
//Suppose we have ten constructor function having one common method
```

//Duplicating method make no sense and if we want to make change in one method then we have make

//change in all the method that is not feasible

//for simplicity we will keep the method separate from the constructor

// and we call the method by passing object as an argument

```
function StudentCall(firstName,lastName)
```

```
{  
  this.firstName=firstName;  
  this.lastName=lastName;  
}
```

//Now we have create separate function

```
const greet=function()  
{  
  console.log(`${this.firstName} ${this.lastName}`);  
}
```

```
function EmployeeCall(firstName,lastName)
```

```
{  
  this.firstName=firstName;  
  this.lastName=lastName;  
}
```

//now we have to point my greet function to either student1 object or emp object otherwise

//when we call greet() without parameter it point to global object for example

`greet();` // it will print undefined undefined

//now we use call() method in which we pass object as an argument

```
greet.call(student1);
```

```
greet.call(emp);
```

//We can also pass another argument apart from object in separate function

```
const greet1=function(prefix,suffix)  
{  
  console.log(`${prefix} ${this.firstName} ${this.lastName} ${suffix}`);  
}
```

```
greet1.call(student1,"Hi","- How are you");
```

```
greet1.call(emp,"Mr.","- How are you");
```

```
console.log("calling through apply function");
```

```
greet1.apply(student1,["Hi","- How are you"]);
```

```
greet1.apply(emp,["Mr.","- How are you"]);
```

//we bind the function with particular object and we can execute the function later

```
console.log("Using bind method");
```

```
let fn=greet1.bind(student1);
```

```
console.log(fn);
```

```
fn("A","B");
```

apply() function

The apply() method is similar to the call() method

The Difference Between call() and apply()

The difference is:

The call() method takes arguments separately.

The apply() method takes arguments as an array.

The apply() method is very handy if you want to use an array instead of an argument list.

Function binding bind() method

In JavaScript function binding happens using Bind() method. With this method, we can bind an object to a common function, so that the function gives different results when needed. otherwise, it gives the same result or gives an error while the code is executing. We use the Bind() method to call a function with this value, this keyword refers to the same object which is currently selected. In other words, the bind() method allows us to easily set which object will be bound by this keyword when a function or method is invoked.

In javaScript we can do functional programming as well as object oriented programming

Functions are treated as Objects.

- We can store them in a variable.
- We can pass them as an arguments.
- We can return them as a value

A programming language is said to have First-class functions if functions in that language are treated like other variables. So the functions can be assigned to any other variable or passed as an argument or can be returned by another function.

JavaScript treat function as a first-class-citizens. This means that functions are simply a value and are just another type of object.

//let consider an example

```
function add(a,b){  
  return a+b;  
}
```

```
function power(a,b)  
{  
  return Math.pow(a,b);  
}
```

//function expression we store function in a variable

```
let addition=function(a,b){  
  return a+b;  
}
```

```
console.log(addition(4,5));  
addition.xyz="Random";// As function is treated as object we can assign property to it  
console.log(addition.xyz);  
console.log(addition(4,5));
```

Higher Order Function

Function that accepts functions as argument or return functions or both

Callback function

A callback is a function passed as an argument to another function This technique allows a function to call another function (Call me back when you are done)

//let consider an example

```
function add(a,b){  
  return a+b;  
}
```

```
function power(a,b)  
{  
  return Math.pow(a,b);  
}
```

//This is high order function because it take function as input

//The function name could be any thing

//Here fn is a callback function

```
function operation(a,b,fn)  
{  
  let r=fn(a,b);  
  console.log(r);  
}
```

```
}  
//now we call the function by passing the argument and function  
operation(4,5,add);  
operation(2,3,power);
```

//Higher order function returning the function

//Let consider the example

```
function outer()  
{
```

```
  function inner()  
  {
```

```
    console.log("inner");  
  }
```

```
  return inner;  
}
```

//In this function we didn't execute the function just return the function

//now type outer() on console we will get inner function definition (It returning function object)

//here outer() function didn't execute as we don't apply parenthesis in outer()

//now type outer()() on console we get inner function output

```
let result=outer();
```

//Similarly if we type outer on the console it give outer function body but didn't execute it

//now result will point to inner function of outer function to execute inner function we write result();

Utility of Higher order function

//Utility of higher order function

//Let consider the example

```
function Employee(name,age,department)  
{
```

```
  this.name=name;
```

```
  this.age=age;
```

```
  this.department=department;  
}
```

```
let employees=[
```

```
  new Employee("Amit",22,"IT"),
```

```
  new Employee("Gunjan",21,"Sales"),
```

```
  new Employee("Vipul",22,"Mkt"),
```

```
  new Employee("Sanjay",27,"Accounts"),
```

```
  new Employee("Harshit",32,"IT")  
]
```

```
function filterByAge(emps,age)  
{
```

```
let result=[];
for(let i=0;i<emps.length;i++)
{
    if(emps[i].age==age)
    {
        result.push(emps[i]);
    }
}
return result;
}
//console.log(filterByAge(employees,22));
```

//Now if we need filter the employees according to department then we write another function

```
function filterByDept(emps,dept)
{
    let result=[];
    for(let i=0;i<emps.length;i++)
    {
        if(emps[i].department==dept)
        {
            result.push(emps[i]);
        }
    }
    return result;
}
//console.log(filterByDept(employees,"IT"));
```

//Now if we have multiple properties then we have to create multiple filter function

//Now if we see the filter function only condition are different but rest of the function are same

//To prevent duplicate code we write higher order function

```
function filter(emps,fn)
{
    let result=[];
    for(let i=0;i<emps.length;i++)
    {
        if(fn(emps[i]))
        {
            result.push(emps[i]);
        }
    }
    return result;
}
```

```
}  
//Now this function and filter by age function will behave same in this function  
//we are passing employee object to check whether age matched or not  
console.log(filter(employees,function(emp){  
    return emp.age==27  
})  
));  
//Now with a single function we can check multiple properties  
//Now with this function we can check department also  
console.log(filter(employees,function(emp){  
    return emp.department=="IT"  
})  
));
```

Scope

Defines the boundary where we can access the variables and functions defined:

- Block Scope
- Function Scope
- Global Scope

Block Scope

Before ES6 (2015), JavaScript variables had only Global Scope and Function Scope. ES6 introduced two important new JavaScript keywords: `let` and `const`. These two keywords provide Block Scope in JavaScript. Variables declared inside a `{ }` block cannot be accessed from outside the block:

```
let a=20;  
if(a==20)  
{  
    let b=30;  
    const c=40;  
  
    console.log(a);  
    console.log(b);  
    console.log(c);  
}  
//now try to access variable b and c here it will not access here  
// because const and let are block level variable or block scope  
(within curly braces)  
// console.log(b);  
// console.log(c);  
// now define another block
```

```
{  
  let d=50;  
}  
console.log(d);
```

Global JavaScript Variables

Variables declared with the var keyword can NOT have block scope. Variables declared inside a { } block can be accessed from outside the block.

A variable declared outside a function, becomes GLOBAL. A global variable has Global Scope:

All scripts and functions on a web page can access it. Variables declared Globally (outside any function) have Global Scope. Global variables can be accessed from anywhere in a JavaScript program. Variables declared with var, let and const are quite similar when declared outside a block. They all have Global Scope:

```
let carName = "Volvo";  
// code here can use carName  
function myFunction() {  
  // code here can also use carName  
}
```

From example:

```
Js/index.js  
let a=20; // here a is a global variable  
if(a==20)  
{  
  let b=30;  
  const c=40;  
  console.log(a);  
  console.log(b);  
  console.log(c);  
}  
//we can access variable a here  
console.log(a);
```

```
js/index2.js
```

```
console.log("The value of a is "+a);
```

html file

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
</body>
<script src="js/index.js"></script>
<script src="js/index2.js"></script>
</html>
```

In JavaScript, a variable can also be used without declaring it. If a variable is used without declaring it, that variable automatically becomes a global variable.

```
function greet() {
  a = "hello"
}
greet();
console.log(a); // hello
```

In the above program, variable `a` is a global variable. If the variable was declared using `let a = "hello"`, the program would throw an error.

When to Use var, let, or const?

1. Always declare variables
2. Always use const if the value should not be changed
3. Always use const if the type should not be changed (Arrays and Objects)
4. Only use let if you can't use const
5. Only use var if you MUST support old browsers.

Function Scope

JavaScript has function scope: Each function creates a new scope. Variables defined inside a function are not accessible (visible) from outside the function. Variables declared with `var`, `let` and `const` are quite similar when declared inside a function.

```
var x=20;
if(1==1)
{
```

```
    console.log(x);  
    var y=30;  
    console.log(y);  
}  
console.log(y);  
function fn(){  
  
    var p=50;  
}  
//here we can't access p because p is function scope  
console.log(p);
```

Introduction let and var in JavaScript

The var keyword has been used in JavaScript for a long time and is function-scoped. It means that a variable declared using var is accessible within the function in which it is declared, as well as any nested functions.

On the other hand, the let keyword was introduced in the ES6 (ECMAScript 6) version of JavaScript. It is block-scoped, which means that a variable declared using let is only accessible within the block in which it is declared. It is similar to how variables are scoped in other programming languages, such as C++ and Java.

JavaScript let vs var

In JavaScript, let and var are both ways to declare variables. The main difference between the two is the scope of the variables they declare. var declarations are globally scoped or function scoped, while let declarations are block scoped.

It means that a variable declared with var can be accessed anywhere within the current function or global scope, while a variable declared with let can only be accessed within the block it was declared. This can be useful for preventing variable declarations from accidentally colliding or overwriting one another.

For example, if you declare a var variable inside a for loop, it will still be accessible outside the loop. However, if you declare a let variable inside the same for loop, it will only be accessible within the loop itself.

In addition, let declarations cannot be redeclared within the same scope, whereas var declarations can be. This can help prevent errors and bugs in your code.

Overall, let is generally considered a more modern and preferred way to declare variables in JavaScript, as it provides more flexibility and safety than

var. It's important to understand the differences between the two and use them appropriately in your code.

Here is a summary of the main differences between var and let in JavaScript:

	var	let
Scope	Function	Block
Redeclaration	Allowed	Not allowed
Can be used in a for loop (counter is not accessible after the loop)	Yes (counter is accessible after the loop)	No

Main differences between var and let

JavaScript let vs var in Local Scope

In JavaScript, let and var are two ways to declare variables. The main difference between them is their scope, or the region of your code in which they are accessible.

var variable

var is the traditional way to declare variables in JavaScript. It has function scope, which means that the variable is accessible within the function in which it is declared, as well as any functions that are nested within that function. If a var variable is not declared within a function, it is considered to be declared in the global scope, which means it is accessible from anywhere in your code.

Here is an example of a var variable in a local scope:

```
function myFunction() {  
  var message = "Hello, world!";  
  console.log(message); // prints "Hello, world!"  
}  
myFunction();
```

```
console.log(message); // prints "undefined"
```

In this example, the message variable is declared within the myFunction function and is only accessible within that function. If we try to access it outside the function, it will be undefined.

Closures

In JavaScript, closure provides access to the outer scope of a function from inside the inner function, even after the outer function has closed. In other function remember its environment.

For example,

```
// outer function
function greet() {

  // variable defined outside the inner function
  let name = 'Rakesh';
  // inner function
  function displayName() {
    // accessing name variable
    return 'Hi' + ' ' + name;
  }
  return displayName;
}
const g1 = greet();
console.log(g1); // returns the function definition
console.log(g1()); // returns the value
```

For example

```
function outer()
{
  let a=10;
  var b=20;
  let inner=function(){
    console.log("inner",a,b);
  }
  return inner;
}
let result=outer();
console.dir(result);// it will display Closure
```

Difference between console.dir and console.log

Console Object: The Console object provides access to the browser's debugging console. The Console object contains many methods, log() and dir() are most used among.

The console.log() method prints out a toString representation of the object in the console to the user.

Syntax:

```
console.log(object) or console.log("string", object)
```

The console.dir() method outputs the list of object properties of a specified object in the console to the user.

Syntax:

```
console.dir(object)
```

setTimeout()

The setTimeout() method executes a block of code after the specified time.

The method executes the code only once.

The commonly used syntax of JavaScript setTimeout is:

```
setTimeout(function, milliseconds);
```

Its parameters are:

function - a function containing a block of code

milliseconds - the time after which the function is executed

The setTimeout() method returns an intervalID, which is a positive integer.

Example 1: Display a Text Once After 3 Second

```
// program to display a text using setTimeout method
```

```
function greet() {  
    console.log('Hello world');  
}  
setTimeout(greet, 3000);  
console.log('This message is shown first');
```

```
//another example
```

```
function greet() {  
    console.log('Hello world');  
}  
setTimeout(greet, 5000);  
console.log('This message is shown first');
```



```
setTimeout(function(){  
    console.log('JavaScript');
```



```
},3000);
```

The setInterval() method repeats a block of code at every given timing event.

The commonly used syntax of JavaScript setInterval is:

setInterval(function, milliseconds);

Its parameters are:

function - a function containing a block of code

milliseconds - the time interval between the execution of the function

```
// program to display a text using setInterval method
function greet() {
    console.log('Hello world');
}
let time=setInterval(greet, 1000); // this function return a number with the
help of this number we           // can stop the function using clearInterval()

function
clearInterval(time);

// program to stop the setInterval() method after five times
let count = 0;
// function creation
let interval = setInterval(function(){

    // increasing the count by 1
    count += 1;

    // when count equals to 5, stop the function
    if(count === 5){
        clearInterval(interval);
    }
    // display the current time
    let dateTime= new Date();
    let time = dateTime.toLocaleTimeString();
    console.log(time);

}, 2000);
```

filter()

The filter() method creates a new array filled with elements that pass a test provided by a function.

The filter() method does not execute the function for empty elements.

The filter() method does not change the original array.

For example:

```
const isEven=number=>number%2==0;
let arr=[1,2,3,4,5];

let arr2=arr.filter(isEven)
console.log(arr2);
let arr3=arr.filter(number=>number%2==0);
console.log(arr3);
```

```
//program to check age in array is adult or not using filter
let ages=[10,12,19,20];
console.log(ages);
let newarr=ages.filter(checkAdult);
console.log(newarr);

function checkAdult(age)
{
    return age>=18;
}
```

map()

map() creates a new array from calling a function for every array element.

map() does not execute the function for empty elements.

map() does not change the original array.

In map() function we convert array from one form to another

Let consider an example

```
let arr=[1,2,3,4,5];
let arr2=arr.map(number=>number*number);
console.log(arr2);
```

Another Example

```
let ary=[1,2,3,4,5];
console.log(ary);
let newarr=ary.map(test);
console.log(newarr);
function test(n)
{
```

```
        return n*10;
    }

let ary=[
    {fname:"Anil",lname:"Kumar"},
    {fname:"Mohit",lname:"Gupta"},
    {fname:"Vipin",lname:"Das"},
];
console.log(ary);
let newarr=ary.map(test);
console.log(newarr);
function test(x)
{
    return x.fname;
}
```

find()

The find() method returns the value of the first element that passes a test.

The find() method executes a function for each array element.

The find() method returns undefined if no elements are found.

The find() method does not execute the function for empty elements.

The find() method does not change the original array.

For example

```
let arr=[1,2,3,4,5];
console.log(arr.find(number=>number==14));

let ages=[10,12,19,20];
console.log(ages);
let result=ages.find(checkAdult);
console.log(result);
function checkAdult(age)
{
    return age>=18;
}
```

findIndex()

The findIndex() method executes a function for each array element.

The findIndex() method returns the index (position) of the first element that passes a test.

The findIndex() method returns -1 if no match is found.

The findIndex() method does not execute the function for empty array elements.

The findIndex() method does not change the original array.

For example

```
let arr=[1,2,3,4,5];  
console.log(arr.findIndex(number=>number==2));
```

```
let ages=[10,12,19,20];  
console.log(ages);  
let result=ages.findIndex(checkAdult);  
console.log(result);  
function checkAdult(age)  
{  
    return age>=18;  
}
```

For example:

```
function Employee(fname,lname,age)  
{  
    this.firstname=fname;  
    this.lastname=lname;  
    this.age=age;  
}
```

```
let employees=[  
    new Employee("Amit","Singh",22),  
    new Employee("Gunjan","Kumar",21),  
    new Employee("Vipul","Verma",22),  
    new Employee("Sanjay","Kapoor",27),  
    new Employee("Harshit","Jain",32)  
]
```

```
console.log(employees.filter(x=>x.age>22));
```

```
let employee2=employees.map(x=>{
```

```
return new Employee(  
    x.firstname.toUpperCase(),  
    x.lastname.toUpperCase(),  
    x.age  
);  
  
});  
console.log(employee2);
```

Rest Operator

The rest operator in JavaScript allows a function to take an indefinite number of arguments and bundle them in an array, thus allowing us to write functions that can accept a variable number of arguments, irrespective of the number of parameters defined. These types of functions in JavaScript are called variadic functions (i.e. functions of indefinite arity (the number of arguments that a function can take)). The rest operator in JavaScript was introduced in ES6 (or ES2015) to handle function parameters efficiently and easily.

The rest parameters hold all those leftover arguments that are not given a separate parameter name (i.e. formally defined in function expression). By using the rest operator, user can call a function with any number of arguments, no matter how it was defined.

The rest parameter always returns an array so we can use any array methods that the JavaScript provides us on it.

Syntax

The rest operator is denoted by three dots (...) as a prefix followed by the name of the parameter. Let us see the syntax of the rest operator in JavaScript to understand it in a better way:

```
function fname(a,b,...myRest) {  
    //statements  
}
```

Here, a and b are normal parameters which will map to first and second argument respectively. Whereas myRest is the rest parameter, that will contain contains the third, fourth, and fifth ... nth arguments i.e. all the remaining arguments other than the values of a and b.

The rest operator is always given as the last parameter and a function definition can have only one rest operator else the code throws a `SyntaxError` error in both cases.

```
/* error */
function fname(a,...myRest, b) {
  //statements
}
```

```
/* error */
function fname(...firstRest , a ,...secondRest) {
  //statements
}
```

Uncaught `SyntaxError`: Rest parameter must be the last formal parameter
For example:

```
function sum(){
  console.log(arguments);
  let sum=0;
  for(let i in arguments)
  {
    sum=sum+arguments[i];
  }
  console.log(`sum=${sum}`);
}
// sum(20,30,40);
// sum(30,50);
//instead of providing all the numeric values we provide first string value
// and then numeric value
// now instead of providing sum it will concatenate all the values
sum("Rakesh",20,30,40);
//To resolve this issue we provide ES6 rest operator
function sumrest(name,...args){
  console.log(arguments);
  let sum=0;
  console.log(`Welcome ${name}`);
  for(let i in args)
  {
    sum=sum+args[i];
  }
  console.log(`sum=${sum}`);
}
```

```
sumrest("Rakesh", 20, 30, 40);
```

```
function sum(...args) {  
  let sum = 0;  
  for (let arg of args) sum += arg;  
  return sum;  
}
```

```
let x = sum(1, 2, 3, 4);  
console.log(x);
```

Difference Between Rest Parameters and the Arguments Object

Following are some differences between the rest parameters in JavaScript and the arguments object:

- The arguments object is not an actual array, but an array-like object whereas the rest parameters are instances of the array.
- Array methods like push, pop, forEach, etc can be applied to rest parameters directly as arguments are stored in the rest parameter in an array but it throws an error in the case of the arguments object as the arguments object is an array-like object, not an actual array thus, we cannot directly use array properties on it (except length).
- The rest parameter includes only the parameters which are not given a separate name and bundles them into an array, while the arguments object includes all the arguments which are passed to a function and bundled in an array-like object.

Program through arguments

```
function sum() {  
  let sum = 0;  
  for (let arg of arguments) sum += arg;  
  return sum;  
}
```

```
let x = sum(1, 2, 3, 4);  
console.log(x);
```

Spread Operator

The spread operator is a new addition to the features available in the JavaScript ES6 version.

The spread operator ... is used to expand or spread an iterable or an array. For example,

```
const arrNumber=[1,2,3,4];  
const arrValue = ['My', 'name', 'is', 'Ramesh'];
```

```
console.log(arrValue); // ["My", "name", "is", "Ramesh"]
console.log(...arrValue); // My name is Ramesh
console.log(...arrNumber);
```

In this case, the code:

```
console.log(...arrValue)
```

is equivalent to:

```
console.log('My', 'name', 'is', 'Ramesh');
```

```
//Example of spread operator
//To resolve this issue we provide ES6 rest operator
function sumrest(name,...args){
    console.log(arguments);
    let sum=0;
    console.log(`Welcome ${name}`);
    for(let i in args)
    {
        sum=sum+args[i];
    }
    console.log(`sum=${sum}`);
}

let arr=[20,30,40,50];
//we use rest operator in function declaration while we use spread operator in
function call
//syntax of spread remain same as rest operator
sumrest("Rakesh",...arr);
```

Copy Array Using Spread Operator

We can also use the spread syntax ... to copy the items into a single array. For example,

```
const arr1 = ['one', 'two'];
const arr2 = [...arr1, 'three', 'four', 'five'];
console.log(arr2);
```

Clone Array Using Spread Operator

In JavaScript, objects are assigned by reference and not by values. For example,

```
let arr1 = [ 1, 2, 3];
let arr2 = arr1;
console.log(arr1); // [1, 2, 3]
console.log(arr2); // [1, 2, 3]
// append an item to the array
```

```
arr1.push(4);  
console.log(arr1); // [1, 2, 3, 4]  
console.log(arr2); // [1, 2, 3, 4]
```

Here, both variables arr1 and arr2 are referring to the same array. Hence the change in one variable results in the change in both variables.

However, if we want to copy arrays so that they do not refer to the same array, we can use the spread operator. This way, the change in one array is not reflected in the other. For example,

```
let arr1 = [ 1, 2, 3];  
// copy using spread syntax  
let arr2 = [...arr1];  
console.log(arr1); // [1, 2, 3]  
console.log(arr2); // [1, 2, 3]  
// append an item to the array  
arr1.push(4);  
console.log(arr1); // [1, 2, 3, 4]  
console.log(arr2); // [1, 2, 3]
```

Spread Operator with Object

We can also use the spread operator with object literals. For example,

```
const obj1 = { firstName:"Raj", lastName : "Kumar"}  
const obj2={age:25}  
const obj3 = {...obj1, ...obj2};  
console.log(obj3);
```

DOM (Document Object Model)

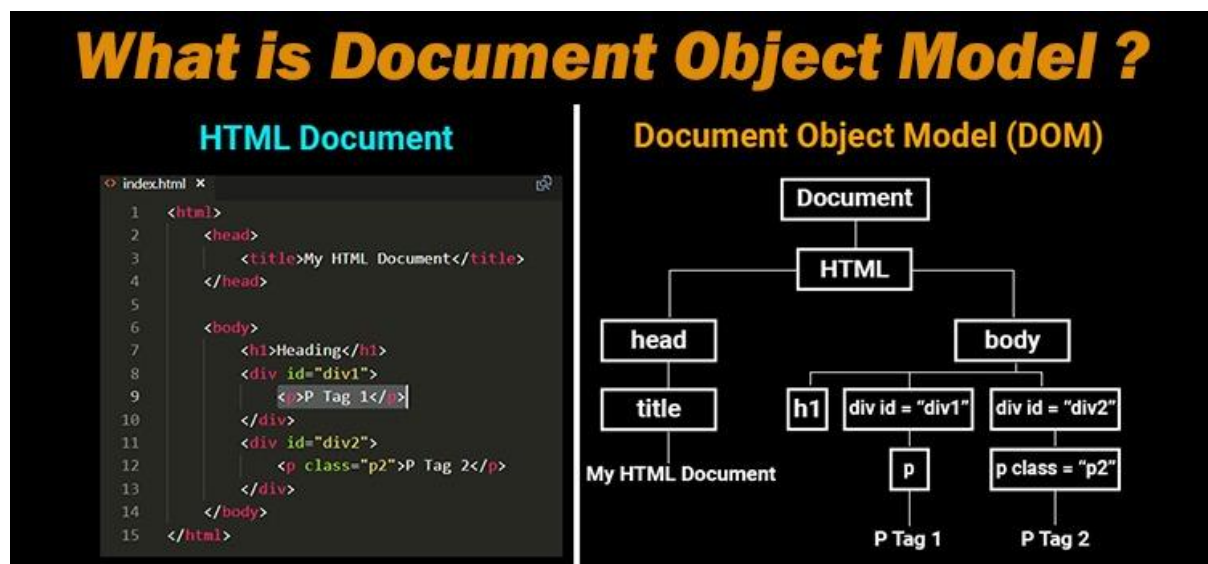
With the HTML DOM, JavaScript can access and change all the elements of an HTML document. When a web page is loaded, the browser creates a **Document Object Model** of the page.

DOM is such a technology with the help of which we can completely control any html document with javaScript.

With the help of DOM, we can access and change any tag , id , class or any attribute.

The **HTML DOM** model is constructed as a tree of **Objects**:

In the Dom tree, the document is the root node or object



Dom tree refers to the HTML page where all the nodes are objects . There can be 3 main types of nodes in the Dom tree:

1. text nodes
2. element nodes
3. comment nodes

A text node is always a leaf of the tree.

Auto Correction

If an erroneous HTML is encountered by the browser, it tends to correct it for example, if we put something after the body, it is automatically moved inside the body. Another example is <table> tag which must contain <tbody>

HTML File

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document object Model</title>
</head>
<body>
  <h1>Heading 1</h1>
  <div id="div1">
    <p>P tag 1</p>
  </div>
  <div id="div2">
    <p class="p2">P tag 2</p>
  </div>
</body>
</html>
```

In inspect element in console type

document (it will display full html structure start with doctype)

document.documentElement(it will display html structure)

document.head (it will display head tag)

document.title= "My title " // it will change the title of the page and
document.title will return the string while other will return object

document.body (to get the body element)

for example

```
document.body.style.backgroundColor="Blue";
//it will set the background color of the body to blue
```

document.body can sometimes be null if the javascript is written before the body tag

For example

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document object Model</title>
</head>
<script>
  console.log(document.body)
</script>
<body>
  <h1>Heading 1</h1>
  <div id="div1">
    <p>P tag 1</p>
  </div>
  <div id="div2">
    <p class="p2">P tag 2</p>
  </div>
</body>
</html>
```

It will display null

This is because we access the `console.log(document.body)` before the `<body>` tag here it does not know what is body tag

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

The HTML DOM is a standard **object** model and **programming interface** for HTML. It defines:

- The HTML elements as **objects**
- The **properties** of all HTML elements
- The **methods** to access all HTML elements
- The **events** for all HTML elements

In other words: **The HTML DOM is a standard for how to get, change, add, or delete HTML elements.**

In the Dom, all HTML elements are defined as objects, so it will have both property and method

The document object represent the web page

If we want to access any element in an HTML page, we always start with accessing the document model

Methods to Select

1. getElementById
2. getElementsByClassName
3. getElementsByTagName
4. querySelector
5. querySelectorAll

getElementById()

The getElementById() method returns an element with a specified value.

The getElementById() method returns null if the element does not exist.

The getElementById() method is one of the most common methods in the HTML DOM. It is used almost every time you want to read or edit an HTML element.

Note

Any id should be unique, but:

If two or more elements with the same id exist, getElementById() returns the first.

For example

```
<body>
  <h1>The Document Object</h1>
  <h2>The getElementById() Method</h2>
  <p id="demo">Random Text</p>
  <script>
    document.getElementById("demo").innerText = "Hello World";
  </script>
</body>
```

innerText

The innerText property sets or returns the text content of an element.

For example:

```
<h1>The Element Object</h1>
  <h2>The innerText Property</h2>
  <button id="myBtn">Click Me</button>
  <p>The text content of the button element is:</p>
  <p id="demo"></p>
  <script>
    let text = document.getElementById("myBtn").innerText;
    document.getElementById("demo").innerText = text;
  </script>
```

Get the element and change its color:

```
<h1 id="demo">The Document Object</h1>
<h2>The getElementById() Method</h2>
<script>
  const myElement = document.getElementById("demo");
  myElement.style.color = "red";
</script>
```

Or

```
<h1 id="demo">The Document Object</h1>
<h2>The getElementById() Method</h2>
<script>
document.getElementById("demo").style.color = "red";
</script>
```

Another Example

```
<body>
  <button onclick="buttonClicked()">Click Me</button>
  <h2 id="heading2">Some Random Text</h2>
</body>
<script>
function buttonClicked()
{
  //alert("button clicked");
  //document.getElementById("heading2").innerHTML="Computer Learning";
  let h= document.getElementById("heading2");
  h.innerText="Computer Learning";
}
</script>
```

Let's see the simple example of document.getElementById() method that prints cube of the given number.

```
<body>
  <form>
    Enter No:<input type="text" id="number" name="number"/><br/>
    <input type="button" value="cube" onclick="getcube()"/>
  </form>
</body>
<script>
function getcube(){
let txtNumber=document.getElementById("number");
let number=txtNumber.value;
alert(number*number*number);
}
</script>
```

Taking Input from user

```
<body>
  <input id="text1" placeholder="username">
```

```
<br/>
<button onclick="fn1()">Click Me</button>
</body>
<script>
function fn1()
{
    //alert("button clicked");
    var str=document.getElementById("text1").value;
    alert("Value inside textbox is :"+str);
}
</script>
```

Dom Manipulation

DOM manipulations are when we use javascript code perform some activity on the HTML document. JS uses the DOM to access different elements on the website and perform specific tasks like - validations, changes, animations, functions etc. HTML events are "things" that happen to HTML elements. When JavaScript is used in HTML pages, JavaScript can "react" on these events.

getElementById() - The getElementById() method returns the element that has the ID attribute with the specified value. This method is one of the most common methods in the HTML DOM, and is used almost every time you want to manipulate, or get info from, an element on your document. Returns null if no elements with the specified ID exists. An ID should be unique within a page. However, if more than one element with the specified ID exists, the getElementById() method returns the first element in the source code.

```
<html>
```

```
  <head>
```

```
    <title>Get element by id</title>
```

```
    <script>
```

```
      function buttonClicked()
```

```
      {
```

```
        //alert("button clicked");
```

```
        document.getElementById("heading2").innerHTML
        ="Computer Learning";
    }
</script>
</head>
<body>
    <button onclick="buttonClicked()">Click
    Me</button>
    <h2 id="heading2">Some Random
    Text</h2>
</body>
</html>
```

Taking Input from user

```
<html>
    <head>
        <title>Take input from the user</title>
        <script>
            function fn1()
            {
                //alert("button clicked");
                var str=document.getElementById("text1").value;
```

```
        alert("Value inside textbox is :"+str);
    }
</script>
</head>
<body>
    <input id="text1" placeholder="username">
    <br/>
    <button onclick="fn1()">Click Me</button>

</body>
</html>
```

Taking Input from Radiobuttons

```
<html>

  <head>

    <title>Take input from the radiobuttons</title>

    <script>

      function fn1()

      {

        var rd1=document.getElementById("rd1");
        var rd2=document.getElementById("rd2");
        if(rd1.checked==true)
          alert("Selected gender is :"+rd1.value);
        else if(rd2.checked==true)
          alert("Selected gender is :"+rd2.value);
        else
          alert("No gender is selected");

      }

    </script>

  </head>

  <body>

    <input id="rd1" name="grp1" type="radio"
value="Male">Male</input>

    <br/>

    <input id="rd2" name="grp1" type="radio"
value="Female">Female</input>

    <br/>

    <button onclick="fn1()">Click Me</button>
```

```
</body>
</html>
```

Taking Input from SelectBox

```
<html>
  <head>
    <title>Take input from the select box</title>
    <script>
      function fn1()
      {
        var select=document.getElementById("selectbox");
        alert(select.options[select.selectedIndex].value);
      }
    </script>
  </head>
  <body>
    <select id="selectbox">
      <option value="Apple">Apple</option>
      <option value="Mango">Mango</option>
      <option value="Grapes">Grapes</option>
    </select>
    <br/>
    <button onclick="fn1()">Click Me</button>
```

```
    </body>
</html>
```

Getting and Setting Data through innerHTML

```
<h1>innerHTML vs innerText</h1>
<button onclick="GetData()">Click here </button>
<div id="demo_data">This is the page</div>
<h3>Demo data will taken from here</h3>
<hr/>
<h3> Result will be shown here!</h3>
<p id="result"></p>
</body>
<script>
function GetData(){
let data=document.getElementById("demo_data").innerHTML;
document.getElementById("result").innerHTML=data;

}
</script>
```

Getting and Setting Data through innerText

```
<h1>innerHTML vs innerText</h1>
<button onclick="GetData()">Click here </button>
<div id="demo_data">This is the page</div>
<h3>Demo data will taken from here</h3>
<hr/>
<h3> Result will be shown here!</h3>
<p id="result"></p>
</body>
<script>
function GetData(){
let data=document.getElementById("demo_data").innerText;
document.getElementById("result").innerText=data;

}
```

```
}  
</script>
```

Another Example

```
<body>  
  
<h1>innerHTML vs innerText</h1>  
<button onclick="GetData()">Click here </button>  
<div id="demo_data"><h1>Heading1</h1></div>  
<h3>Demo data will taken from here</h3>  
<hr/>  
<h3> Result will be shown here!</h3>  
<p id="result"></p>  
</body>  
<script>  
  function GetData(){  
    let data=document.getElementById("demo_data").innerText;  
    document.getElementById("result").innerText=data;  
  }  
</script>
```

Another Example using innerHTML

```
<body>  
  
<h1>innerHTML vs innerText</h1>  
<button onclick="GetData()">Click here </button>  
<div id="demo_data"><h1>Heading1</h1></div>  
<h3>Demo data will taken from here</h3>  
<hr/>  
<h3> Result will be shown here!</h3>  
<p id="result"></p>  
</body>  
<script>  
  function GetData(){  
    let data=document.getElementById("demo_data").innerHTML;  
    document.getElementById("result").innerHTML =data;  
  }  
</script>
```

Another Example

```
<body>
  <h1>innerHTML vs innerText</h1>
  <button onclick="GetData()">Click here </button>
  <div id="demo_data"><h1>Heading1</h1>
  <br>
  <br>
  <button>Sample button</button>

  </div>
  <h3>Demo data will taken from here</h3>
  <hr/>
  <h3> Result will be shown here!</h3>
  <p id="result"></p>
</body>
<script>
  function GetData(){
    let data=document.getElementById("demo_data").innerHTML;
    document.getElementById("result").innerHTML=data;
  }
</script>
```

Now use innerText instead of innerHTML and see the result

getElementByTagName method

The `getElementsByTagName()` method returns a collection of all elements in the document with the specified tag name, as a `NodeList` object. The `NodeList` object represents a collection of nodes. The nodes can be accessed by index numbers. The index starts at 0.

```
<body>
  <div>Rakesh</div>
  <div>Amit</div>
  <div>Nitin</div>
</body>
<script>
```

```
let tname=document.getElementsByTagName("div");
// console.log(tname);
// console.log(tname[0]);
for(let i=0;i<tname.length;i++)
{
    console.log(tname[i].innerText);
}

</script>

<div id="name">
    <div>Rakesh</div>
    <div>Amit</div>
    <div>Nitin</div>
</div>
<div id="test">
    <div>Test1</div>
    <div>Test2</div>
    <div>Test3</div>
</div>
</body>
<script>
    let divid=document.getElementById("test");
    let tname=divid.getElementsByTagName("div");
    // console.log(tname);
    // console.log(tname[0]);
    for(let i=0;i<tname.length;i++)
    {
        console.log(tname[i].innerText);
    }

</script>

<html>
    <head>
        <title>Take input from the select box</title>
        <script>
            function fn1()
            {
```

```
        var para=document.getElementsByTagName("p");
        para[0].style.fontSize="25px";
        para[1].style.color="red";
        para[2].style.fontWeight="bold";
        para[3].style.fontStyle="italic";

        for(var i=0;i<para.length;i++)
        {
            para[i].style.fontSize="25px";
        }
    }
</script>
</head>
<body>
    <p>This is paragraph 1</p>
    <p>This is paragraph 2</p>
    <p>This is paragraph 3</p>
    <p>This is paragraph 4</p>
    <p>This is paragraph 5</p>
    <br/>
    <button onclick="fn1()">Click Me</button>

</body>
</html>
```

eval()

The eval() method evaluates or executes an argument. If the argument is an expression, eval() evaluates the expression.

For example:

```
let str="20 * 3";  
let res=eval(str);  
console.log("Result=",res);
```

slice(0,-1)

A negative number selects from the end of the string.

getElementByClassName method

The `getElementsByClassName()` method returns a collection of all elements in the document with the specified class name, as a `NodeList` object. The `NodeList` object represents a collection of nodes. The nodes can be accessed by index numbers. The index starts at 0.

```
<body>
  <div class="name">Rakesh</div>
  <div class="name">Amit</div>

</body>
<script>
  let cname=document.getElementsByClassName("name");
  // console.log(cname);
  // console.log(cname[0]);
  for(let i=0;i<cname.length;i++)
  {
    console.log(cname[i]);
  }
</script>
```

```
<html>

  <head>

    <title>getElementsByClassName</title>

    <script>

      function fn1()

      {

        var
element=document.getElementsByClassName("mypara");

        for(var i=0;i<element.length;i++)

        {

          element[i].style.color="red";
```

```
        }

    }

</script>

</head>

<body>

    <p class="mypara">This is paragraph 1</p>

    <p>This is paragraph 2</p>

    <p class="mypara">This is paragraph 3</p>

    <p>This is paragraph 4</p>

    <p class="mypara">This is paragraph 5</p>

    <a href="#" class="mypara">This is an anchor tag</a>

    <br/>

    <button onclick="fn1()">Click Me</button>


</body>

</html>
```

querySelector()

The `querySelector()` method returns the first element that matches a CSS selector.

To return all matches (not only the first), use the `querySelectorAll()` instead.

```
<body>
  <h1 id="head1">First Heading</h1>
  <h2 class="intro">heading 1</h2>
  <h2 class="intro">heading 2</h2>
  <div id="div1">
    <p class="intro">Para 1</p>
    <p>Para 2</p>
```

```
<p class="intro">Para 3</p>
<p class="intro">Para 4</p>
<p>Para 25</p>
<p class="intro">Para 6</p>
<p class="intro">Para 7</p>
<p>Para 8</p>
<p class="intro">Para 9</p>
</div>
</body>
<script>
  // let elmHead=document.querySelector("#head1");
  // elmHead.style.color='red';
  // let elmHead2=document.querySelector(".intro");
  // elmHead2.style.color='blue';
  // let elmp=document.querySelector("p:last-child");
  // elmp.style.color='blue';
  // let elmp=document.querySelector("p:nth-child(2)");
  // elmp.style.color='red';
  // elmp.style.backgroundColor='yellow';
  // let elmp=document.querySelectorAll("p");
  // elmp[0].style.color='red';
  // elmp[1].style.backgroundColor='yellow';
  let oddp=document.querySelectorAll("p:nth-child(odd)");
  for(let i=0;i<oddp.length;i++)
  {
    oddp[i].style.backgroundColor='#f4f4f4';
  }
  let evenp=document.querySelectorAll("p:nth-child(even)");
  for(let i=0;i<evenp.length;i++)
  {
    evenp[i].style.backgroundColor='#ccc';
  }
  // let elm=document.querySelector("p.intro");
  // console.log(elm);
  // elm.innerText="Hello";
  // let elm=document.querySelectorAll("p.intro1");
  // console.log(elm);

  // for(let i=0;i<elm.length;i++)
  // {
  //   console.log(elm[i].innerText);
  // }
</script>
```

Create and Append Elements

```
<body>
  <ul id="box">
    <li>Amit</li>
    <li>Rakesh</li>
    <li>Nitin</li>
    <li>Pankaj</li>
  </ul>
  <input type="text" id="name">
  <button onclick="addNewLi()">Add More...</button>
</body>
```

```
<script>
  function addNewLi()
  {
    let ulBox=document.getElementById("box");
    let newli=document.createElement("li");
    newli.className="new_li abc";
    newli.id="liData";
    let name=document.getElementById("name").value;
    newli.innerText=name;
    // newli.textContent=name;
    //let liText=document.createTextNode(name)
    // let liText=document.createTextNode("Bhavesb");
    ///newli.appendChild(liText);
    ulBox.appendChild(newli);
    document.getElementById("name").value="";
    document.getElementById("name").focus();
  }
</script>
```

createElement()

The createElement() method creates an element node.

Create a <p> element and append it to the document:

```
const para = document.createElement("p");

para.innerText = "This is a paragraph";
```

```
document.body.appendChild(para);
```

Create a <p> element and append it to an element:

```
const para = document.createElement("p");  
  
para.innerHTML = "This is a paragraph."  
  
document.getElementById("myDIV").appendChild(para);
```

createTextNode()

The createTextNode() method creates a text node.

Create a text node and append it to the document:

```
let textNode = document.createTextNode("Hello World");  
document.body.appendChild(textNode);
```

Create a <h1> element with a text node:

```
const h1 = document.createElement("h1");  
const textNode = document.createTextNode("Hello World");  
h1.appendChild(textNode);
```

Create a <p> element with a text node:

```
const para = document.createElement("p");  
const textNode = document.createTextNode("Hello World");  
para.appendChild(textNode);
```

appendChild()

The appendChild() method appends a node (element) as the last child of an element.

Append an item to a list:

```
const node = document.createElement("li");  
const textnode = document.createTextNode("Water");
```

```
node.appendChild(textnode);  
document.getElementById("myList").appendChild(node);
```

textContent

The textContent property sets or returns the text content of the specified node

Traversing Elements

```
<body>  
  <ul id="box">  
    <li>Amit</li>  
    <li>Rakesh</li>  
    <li>Nitin</li>  
  </ul>  
  <ul>  
    <li>Data1</li>  
    <li id="ldata">Data2</li>  
    <li>Data3</li>  
  </ul>  
  <!-- <input type="text" id="name">  
  <button onclick="addNewLi()">Add More...</button> -->  
</body>  
<script>  
  //Traversing Elements  
  let ulBox=document.getElementById("box");  
  console.log(ulBox);  
  let firstelmChild=ulBox.firstElementChild;  
  console.log(firstelmChild);  
  let lastelmChild=ulBox.lastElementChild;  
  console.log(lastelmChild);  
  //To get all the childrens  
  console.log(ulBox.children);  
  console.log(ulBox.children[0]);  
  let liBox=document.getElementById("ldata");  
  console.log(liBox.parentElement);  
  console.log(liBox.nextElementSibling);  
  console.log(liBox.previousElementSibling);  
</script>
```

firstElementChild

The firstElementChild property returns the first child element of the specified element.

lastElementChild

The lastElementChild property returns the last child element of an element.

Children

The children property returns a collection of an element's child elements.

parentElement

The parentElement property returns the parent element of the specified element

nextElementSibling

The nextElementSibling property returns the next element in the same tree level.

previousElementSibling

The previousElementSibling property returns the previous element in the same tree level.

Remove Elements

```
<body>
  <ul id="box">
    <li>Amit</li>
    <li>Rakesh</li>
    <li>Nitin</li>
    <li>Pankaj</li>
  </ul>
  <ul id="box1">
    <li id="li1">Para1</li>
    <li id="li2">Para2</li>
    <li id="li3">Para3</li>

  </ul>
  <input type="text" id="name">
  <button onclick="addNewLi()">Add More...</button>
  <button onclick="remove()">Remove</button>
```

```
</body>

<script>
  function remove()
  {
    //remove child
    let ulBox=document.getElementById("box");
    //document.body.removeChild(ulBox);
    let liBox=ulBox.firstElementChild;
    //ulBox.removeChild(liBox);
    let liBoxSibling=liBox.nextElementSibling;
    //ulBox.removeChild(liBoxSibling);
    let ulBox1=document.getElementById("box1");
    let liBoxId=document.getElementById("li2");
    ulBox1.removeChild(liBoxId);

  }

  function addNewLi()
  {
    let ulBox=document.getElementById("box");
    let newli=document.createElement("li");
    newli.className="new_li abc";
    newli.id="liData";
    let name=document.getElementById("name").value;
    newli.innerText=name;
    // newli.textContent=name;
    //let liText=document.createTextNode(name)
    // let liText=document.createTextNode("Bhavesb");
    ///newli.appendChild(liText);
    ulBox.appendChild(newli);
    document.getElementById("name").value="";
    document.getElementById("name").focus();
  }
</script>
```

removeChild()

The **removeChild()** method removes an element's child.

Insert Before

```
<body>
  <ul id="box">
    <li>Amit</li>
    <li>Mukesh</li>
    <li>Pankaj</li>
  </ul>
  <button onclick="InsertNewLi()">Insert new</button>

</body>
<script>
  function InsertNewLi()
  {
    let ulBox=document.getElementById("box");
    let newli=document.createElement("li");
    newli.textContent="Gunjan";
    ///let pos=ulBox.firstChild;
    let pos=ulBox.firstChild.nextElementSibling;
    ulBox.insertBefore(newli,pos);
  }
</script>
```

The insertBefore() method inserts a child node before an existing child.

Mouse Events (onclick, ondblclick, onmouseover, onmouseout)

```
<head>
  <style>

    #box{
      height:300px;
      width:300px;
      background:green;
      margin:200px 0 0 125px;
    }
  </style>

</head>
<body>
  <!-- left mouse button clicked
```

```
<div id="box" onclick="fn1()"></div> -->

<!-- right mouse button clicked
<div id="box" oncontextmenu="fn1()"></div> -->

<!-- on double clicked
<div id="box" ondblclick="fn1()"></div> -->

<!-- on mouse down (button remain clicked) -->
<!-- <div id="box" onmousedown="fn1()"></div> -->

<!-- on mouse up (when we release the mouse) -->
<!-- <div id="box" onmouseup="fn1()"></div> -->

<!-- The onmouseover event occurs when the mouse pointer enters an element. -->
<!-- <div id="box" onmouseover="fn1()"></div> -->
<!--
The onmouseout event occurs when the mouse pointer moves out of an element. -->
<div id="box" onmouseout="fn1()"></div>

</body>
<script>
    function fn1()
    {
        alert("Button Clicked");
    }
</script>
```

Change Image onmouseover and onmouseout events in JavaScript

Onmouseover event - The onmouseover event occurs when the mouse pointer is moved onto an element.

Onmouseout event - The onmouseout event occurs when the mouse pointer is moved out of an element

```
<html>
    <head>
        <title>mouseover and mouseout event</title>
        <script>
            function setNewImage()
            {
                document.getElementById("img1").src="image/paris.jpg"
            }
        </script>
    </head>
</html>
```

```
        function setOldImage()
        {
            document.getElementById("img1").src="image/img_tree.jpg";
        }
    </script>
</head>
<body>
    
    </body>
</html>
```

Keyboard Events (onkeydown, onkeyup, onkeypress)

```
<head>
<style>

    #form{
        width:250px;
        border:2px solid gray;
        margin:150px auto 0 auto;
        padding:25px;
        border-radius:15%;
        background:rgba(0,0,0,0.5);
        color:#fff;
        box-shadow: -2px 2px 15px #000;

    }
    p,input{
        font-size:20px;

    }
    input{
        width:225px;
        border-radius:10px;
        border:1px solid #000;
        padding:8px;
        background:#fff;
        box-shadow:inset -2px 2px 10px #000;
    }
</style>

</head>
<body>
```

```
<div id="form">
  <h1 id="h1">Keyboard Events</h1>
  <!-- The onkeydown event occurs when the user presses a key on the keyboard. -->
  <p>Onkeydown</p>
  <!-- <input type="text" onkeydown="alert(this.value);" -->
  <input type="text"
onkeydown="document.getElementById('form').style.background=this.value;">
  <!-- The onkeyup event occurs when the user releases a key on the keyboard. -->
  <p>Onkeyup</p>
  <!-- <input type="text" onkeyup="alert(this.value);" -->
  <input type="text"
onkeyup="document.getElementById('form').style.background=this.value;">
  <p>Onkeypress</p>
  <!-- The onkeypress event occurs when the user presses a key on the keyboard. -->
  <input type="text" onkeypress="alert(this.value);">
</div>

</body>
```

The onkeypress event is deprecated.

It is not fired for all keys (like ALT, CTRL, SHIFT, ESC) in all browsers.

To detect if the user presses a key, always use the onkeydown event. It works for all keys.

```
<body>
  <button onclick="openWindow()">Open Window</button>
  <button onclick="closeWindow()">Close Window</button>

</body>
<script>
  var windowData;
  function openWindow()
  {
    windowData= window.open("https://www.google.com/", "",
"width=500,height=500,left=200px,top=200px");
  }
  function closeWindow()
  {
    windowData.close();
  }
</script>
```

Form Validations in JavaScript

Sample.html

```
<html>

  <head>

    <title>form Validation</title>

    <script>

      function validate()

      {

        var username=document.getElementById("uname");

        var password=document.getElementById("pass");

        if(username.value.trim()==""||password.value.trim()=="")

        {

          alert("No blank field allowed");

          return false;

        }

        else

        {

          return true;

        }

      }

    </script>

  </head>

  <body>

    <form action="message.html" onsubmit="return validate()">

      <input type="text" id="uname" placeholder="username"/><br><br>

      <input type="password" id="pass" placeholder="password"/><br><br>

      <button type="submit">Submit</button>

    </form>

  </body>

</html>
```

```
        </form>

    </body>

</html>

Message.html.

<html>

    <head>

        <title>form Validation</title>

        <script>


        </script>

    </head>

    <body>

        <h1>Welcome</h1>

    </body>

</html>
```

Form Validations in JavaScript

Check the length and Add style also

```
<html>

    <head>

        <title>form Validation</title>

        <script>

            function validate()

            {

                var

username=document.getElementById("uname");

                var

password=document.getElementById("pass");
```

```
        if(username.value.trim()=="")
        {
            //alert("Blank Username");
            username.style.border="solid 3px red";

document.getElementById("lblUser").style.visibility="visible";

            return false;
        }
        else if (password.value.trim()=="")
        {
            password.style.border="solid 3px red";

document.getElementById("lblPass").style.visibility="visible";

document.getElementById("lblPass").innerHTML="invalid";

            return false;
        }
        else if(password.value.length<5)
        {
            password.style.border="solid 3px red";

document.getElementById("lblPass").style.visibility="visible";

document.getElementById("lblPass").innerHTML="length is
short";

            return false;
        }
        else
        {
            return true;
        }
    }
}
```

```
        }  
    }  
  
    </script>  
  
    </head>  
  
    <body>  
        <form action="message.html" onsubmit="return  
validate()">  
            <input type="text" id="uname"  
placeholder="username"/>  
            <label id="lblUser"  
style="color:red;visibility:hidden">invalid</label>  
            <br><br>  
            <input type="password" id="pass"  
placeholder="password"/>  
            <label id="lblPass"  
style="color:red;visibility:hidden">invalid</label>  
            <br><br>  
            <button type="submit">Submit</button>  
        </form>  
    </body>  
</html>
```

Regular Expressions in JavaScript

```
<html>

  <head>

    <title>Regular Expression In JS</title>

    <script>

      function validate()

      {

        var
username=document.getElementById("uname").value;

        var regx=/^E00[a-zA-Z]*/

        if(regx.test(username))

        {

          alert("Valid username");

        }

        else

        {

          alert("Invalid Username");

        }

        document.getElementById("lblUser").style.visibility="visible";

      }

    </script>

  </head>

  <body>

    <form action="message.html" >
```

```
        <input type="text" id="uname"
placeholder="username"/><br/>

        <label id="lblUser"
style="color:red;visibility:hidden">invalid Username</label>

        <br><br>

        <button onclick="validate()"
type="button">Submit</button>

    </form>

</body>

</html>
```

Mobile Number Validation in JavaScript using Regular Expression

```
<html>
  <head>
    <title>Regular Expression In JS</title>
    <script>
      function validate()
      {
        var mobile=document.getElementById("mob").value;
        var regx=/^[7-9]\d{9}$/
        if(regx.test(mobile))
        {
          document.getElementById("lblText").innerHTML="Valid";
          document.getElementById("lblText").style.visibility="visib
le";

          document.getElementById("lblText").style.color="green";
        }
        else
        {
          document.getElementById("lblText").innerHTML="Invalid";
          document.getElementById("lblText").style.visibility="visib
le";

          document.getElementById("lblText").style.color="red";
        }
      }
    </script>
  </head>
  <body>
```

```
<form action="message.html" >
  <input type="text" id="mob" placeholder="mobile no"/><br/>
  <label id="lblText" style="color:red;visibility:hidden">invalid
Username</label>
  <br><br>
  <button onclick="validate()" type="button">Submit</button>
</form>
</body>
</html>
```

Email Validation in JavaScript

- + minimum one and more character
- ? make the characters optional

```
<html>
  <head>
    <title>Email Validation In JS</title>
    <script>
      function validate()
      {
        var email=document.getElementById("email").value;
        var regx=/^([a-zA-Z0-9\.-]+)@([a-zA-Z0-9-]+).([a-z]{2,8})(.[a-
z]{2,8})?$/
        if(regx.test(email))
        {
          document.getElementById("lblText").innerHTML="Valid";
          document.getElementById("lblText").style.visibility="visib
le";
          document.getElementById("lblText").style.color="green";
        }
        else
        {
          document.getElementById("lblText").innerHTML="Invalid";
          document.getElementById("lblText").style.visibility="visib
le";
          document.getElementById("lblText").style.color="red";
        }
      }
    </script>
  </head>
</html>
```

```
    }  
  
    </script>  
</head>  
<body>  
    <form action="message.html" >  
        <input type="text" id="email" placeholder="email"/><br/>  
        <label id="lblText" style="color:red;visibility:hidden">invalid  
Username</label>  
        <br><br>  
        <button onclick="validate()" type="button">Submit</button>  
    </form>  
</body>  
</html>
```

Password Matching using JavaScript

```
<html>
  <head>
    <script>

      // Function to check Whether both passwords
      // is same or not.
      function checkPassword() {
        var pass= document.getElementById("pass");
        var cpass = document.getElementById("cpass");

        // If password not entered
        if (pass.value.trim() == '')
        {
          alert ("Please enter Password");
          pass.focus();
          return false;
        }

        // If confirm password not entered
        else if (cpass.value.trim() == '')
        {
          alert ("Please enter confirm password");
          cpass.focus();
          return false;
        }

        // If Not same return False.
        else if (pass.value.trim() != cpass.value.trim()) {
          alert ("\nPassword did not match: Please try again...")
          pass.focus();
          return false;
        }

        // If same return True.
        else{
          alert("Password Match: Welcome ")
          return true;
        }
      }
    </script>

  </head>
</body>
```

```
<form onSubmit = "return checkPassword()">
<table border = 1 align = "center">
  <tr>
    <!-- Enter Username -->
    <td>Username:</td>
    <td><input type = "text" id = "uname" name = "uname" ></td>
  </tr>
  <tr>
    <!-- Enter Password. -->
    <td>Password:</td>
    <td><input type = "password" name = "pass" id = "pass" ></td>
  </tr>
  <tr>
    <!-- To Confirm Password. -->
    <td>Confirm Password:</td>
    <td><input type = "password" name = "cpass" id = "cpass" ></td>
  </tr>
  <tr>
    <td colspan = 2 align = right>
      <input type = submit value = "Submit"></td>
    </tr>
</table>
</form>
</body>
</html>
```

Form Validation Example

```
<!DOCTYPE html>
<html>

<body>
  <form action="message.html" method="post" onsubmit="return validate()">
    <div class="form-group">
      <label for="first_name">First Name</label>
      <input type="text" class="form-control" id="first_name" name="first_name"
placeholder="Enter first Name">
    </div>
    <div class="form-group">
      <label for="last_name">Last Name</label>
      <input type="text" class="form-control" id="last_name" name="last_name"
placeholder="Enter last Name">
    </div>
    <div class="form-group">
      <label for="salary">Salary</label>
      <input type="number" class="form-control" id="salary" name="salary"
placeholder="Enter Salary">
    </div>
    <div class="form-group">
      <label for="dept">Department</label>
      <select class="custom-select" name="dept" id="dept">
        <option selected disabled="true" value="">Choose...</option>
        <option value="1">Data</option>
      </select>
    </div>
    <div class="form-group">
      <label for="role">Role</label>
      <select class="custom-select" name="role" id="role">
        <option selected disabled="true" value="">Choose...</option>
        <option value="1">Data</option>
      </select>
    </div>
    <div class="form-group">
      <label for="dept">Phone Number</label>
      <input type="number" class="form-control" id="phone" name="phone"
placeholder="Enter Phone">
    </div>
    <div class="form-group">
      <label for="dept">Bonus</label>
      <input type="number" class="form-control" id="bonus" name="bonus"
placeholder="Enter Bonus">
    </div>
  </form>
</body>
</html>
```

```
<button type="submit" class="btn btn-primary">Submit</button>
</form>
```

```
</body>
```

```
<script>
```

```
function validate() {
    fname = document.getElementById("first_name");
    lname = document.getElementById("last_name");
    sal = document.getElementById("salary");
    dept = document.getElementById("dept");
    role = document.getElementById("role");
    ph = document.getElementById("phone");
    phdata = document.getElementById("phone").value;
    bonus = document.getElementById("bonus");
    let result = true;
    var regx = /^[6-9]\d{9}$/
    if (fname.value.trim() == "") {
        alert("first name cannot be blank");
        fname.focus();
        result = false;
    }
    if (lname.value.trim() == "") {
        alert("last name cannot be blank");
        lname.focus();
        result = false;
    }
    if (salary.value.trim() == "") {
        alert("Salary cannot be blank");
        salary.focus();
        result = false;
    }
    if (dept.value.trim() == "") {
        alert("Select the department!!!");
        dept.focus();
        result = false;
    }
    if (role.value.trim() == "") {
        alert("Select the role!!!");
        role.focus();
        result = false;
    }
    if (ph.value.trim() == "") {
        alert("Phone cannot be blank");
        ph.focus();
        result = false;
    }
    if (!regx.test(phdata)) {
```

```
        alert("Invalid ph no");
        ph.focus();
        result = false;
    }
    if (bonus.value.trim() == "") {
        alert("Bonus cannot be blank");
        bonus.focus();
        result = false;
    }
    if (result == true) {
        alert("Data is valid");
        return true;
    }
    else {
        return false;
    }
}
</script>

</html>
```

Program to check string is Palindrome or not

```
<html>

<head>
    <title> JavaScript Palindrome </title>
</head>

<body>

<!-- Use JavaScript programming code to validate the Palindrome numbers or strings. -->
<script>
    function palindromeFun(str) {
        // convert the string into an array using the string.split() function
        const arrValue = string.split(""); //

        // use reverse() method to reverse the array values
        const reveArrVal = arrValue.reverse();

        // use join() method to group the array values into the string
        const revString = reveArrVal.join("");

        if (string == revString) // if string condition is equal to the revString
```

```
{
  alert('It is a Palindrome string '); // print the Palindrome
}
else {
  alert(' It is not a Palindrome string'); // if the condition is not true.
}
}
// take a string from the user
const string = prompt(' Enter the string to check Palindrome ');
const value = palindromeFun(string); // call the function
console.log(value);
</script>
</body>

</html>
```

JavaScript Date

```
<body>
  <h2>JavaScript Dates</h2>
<script>
//It take date from client machine not from the server
const d = new Date();
console.log(d);
console.log(d.toString());
console.log("Date:- "+d.getDate());
console.log("Full Year:- "+d.getFullYear());
console.log(`Month :-${d.getMonth()}`);
console.log(`Day :-${d.getDay()}`);
console.log(`Hours :-${d.getHours()}`);
console.log(`Minutes :-${d.getMinutes()}`);
console.log(`Seconds :-${d.getSeconds()}`);
console.log(`Milliseconds :-${d.getMilliseconds()}`);
const newDate=new Date('Decemeber 30 2023')
console.log(newDate);
console.log(newDate.toString());
console.log("Date:- "+newDate.getDate());
console.log("Full Year:- "+newDate.getFullYear());
console.log(`Month :-${newDate.getMonth()}`);
console.log(`Day :-${newDate.getDay()}`);
console.log(`Hours :-${newDate.getHours()}`);
console.log(`Minutes :-${newDate.getMinutes()}`);
console.log(`Seconds :-${newDate.getSeconds()}`);
console.log(`Milliseconds :-${newDate.getMilliseconds()}`);
const sDate=new Date();
sDate.setDate(5);
sDate.setMonth(1);
sDate.setFullYear(2024);
```

```
sDate.setHours(15);
console.log(`Date :-${sDate}`)
const curDate=new Date();
console.log(`Current Date
${curDate.getDate()}/${curDate.getMonth()}/${curDate.getFullYear()}`);
</script>
```

In JavaScript, date and time are represented by the Date object. The Date object provides the date and time information and also provides various methods.

A JavaScript date defines the EcmaScript epoch that represents milliseconds since 1 January 1970 UTC. This date and time is the same as the UNIX epoch (predominant base value for computer-recorded date and time values).

new Date()

We can create a date object using the new Date() constructor. For example,

```
const timeNow = new Date();
console.log(timeNow); // shows current date and time
```

new Date(milliseconds)

The Date object contains a number that represents milliseconds since 1 January 1970 UTC.

new Date(milliseconds) creates a new date object by adding the milliseconds to the zero time. For example,

```
const time1 = new Date(0);
// epoch time
console.log(time1); // Thu Jan 01 1970 05:30:00
// 1000000000000 milliseconds after the epoch time
const time2 = new Date(1000000000000)
console.log(time2); // Sat Mar 03 1973 15:16:40
```

Note: 1000 milliseconds is equal to 1 second.

new Date(date string)

new Date(date string) creates a new date object from a date string.

In JavaScript, there are generally three date input formats.

ISO Date Formats

We can create a date object by passing ISO date formats. For example,

// ISO Date(International Standard)

```
const date = new Date("2020-07-01");
// the result date will be according to UTC
console.log(date); // Wed Jul 01 2020 05:45:00 GMT+0545
```

Short and Long date format

The other two date formats are short date format and long date format.

```
// short date format "MM/DD/YYYY"
const date = new Date("03/25/2015");
console.log(date); // Wed Mar 25 2015 00:00:00 GMT+0545
// long date format "MMM DD YYYY"
const date1 = new Date("Jul 1 2020");
console.log(date1); // Wed Jul 01 2020 00:00:00 GMT+0545
// month and day can be in any order
const date2 = new Date("1 Jul 2020");
console.log(date2); // Wed Jul 01 2020 00:00:00 GMT+0545
```

`now()` Returns the numeric value corresponding to the current time (the number of milliseconds elapsed since January 1, 1970 00:00:00 UTC)

`getFullYear()` Gets the year according to local time

`getMonth()` Gets the month, from 0 to 11 according to local time

`getDate()` Gets the day of the month (1–31) according to local time

`getDay()` Gets the day of the week (0-6) according to local time

`getHours()` Gets the hour from 0 to 23 according to local time

`getMinutes` Gets the minute from 0 to 59 according to local time

`setFullYear()` Sets the full year according to local time

`setMonth()` Sets the month according to local time

`setDate()` Sets the day of the month according to local time

Date Methods

1. `toString()`
2. `getSeconds()`
3. `getMilliseconds()`
4. `setHours()`
5. `setMilliseconds()`
6. `setMinutes()`
7. `setSeconds()`

Date Object

The Date object works with dates and times. Date objects are created with `new Date()`. There are four ways of instantiating (creating) a date:

<h2>JavaScript Dates</h2>

<p>new Date() creates a new date object with the current date and time:</p>

<p id="demo"></p>

<script>

const d = new Date();

document.getElementById("demo").innerHTML = d;

</script>

Countdown Formula

```
let d1 = new Date("1-04-2024");
let d2 = new Date();
let diff = d2 - d1;
let daydiff = Math.floor(diff / 1000 / 60 / 60 / 24);
let hrs = Math.floor(diff / 1000 / 60 / 60);
let remHrs=hrs%24;
let min = Math.floor(diff / 1000 / 60);
let remMins=min%60;
let sec = Math.floor(diff / 1000);
let remSecs=sec%60;
console.log(`Days:-${daydiff}`);
console.log(`Hrs:-${hrs}`);
console.log(`Remaining Hrs:-${remHrs}`);
console.log(`Min:-${min}`);
console.log(`Remaining Min:-${remMins}`);
console.log(`Sec:-${min}`);
console.log(`Remaining Sec:-${remSecs}`);
```